

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

Multimodal Autonomous Rover for Agricultural Monitoring

EE 175AB Final Report
Department of Electrical Engineering, UC Riverside

Project Team Member(s)	Adam Lachman, Angel Pulido, Ying Gao
Date Submitted	3/17/2026
Section Professor	Roman Chomko, Md Shazid Islam
Revision	1.0
URL of Project YouTube Videos, Wiki/Webpage	https://github.com/angelS-217/Multimodal-Autonomous-Rover-for-Precision-Agricultural-Analysis/tree/main
Permanent Emails of all team members	alach005@ucr.edu , jpuli018@ucr.edu , ygao243@ucr.edu

Summary

This report presents design, implementation, testing, and documentation procedures for our hybrid agricultural rover used in precision crop monitoring. The rover performs manual navigation through the use of a gamepad controller as well as autonomous driving using LiDAR scanning. Through integration of computer vision, the rover leverages PlantCV software to analyze plant health in real-time using images captured by the onboard camera. All onboard functionality occurs on the Raspberry Pi 5 while commands are sent between the rover and base station through ESP32 LoRa wireless communication. Our rover enables the targeted intervention of plants suffering from nutrient deficiencies and disease while reducing harmful chemical runoff and waste of resources.

Team 24-Sec.022 <u>Dept. of Electrical and Computer Engineering, UCR</u>	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

Team 24-Sec.022 <u>Dept. of Electrical and Computer Engineering, UCR</u>	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring Ver. 1.0, Mar. 2026
--	---

Revisions

Version	Description of Version	Author(s)	Date Completed	Approval
1.0	Complete Final Report	Adam Lachman, Angel Pulido, Ying Gao	03/17/2026	✓

Table of Contents

REVISIONS	3
TABLE OF CONTENTS	4
1 * EXECUTIVE SUMMARY	6
2 * INTRODUCTION	7
2.1 * DESIGN OBJECTIVES AND SYSTEM OVERVIEW	7
2.2 * BACKGROUNDS AND PRIOR ART	8
2.3 * DEVELOPMENT ENVIRONMENT AND TOOLS	8
2.4 * RELATED DOCUMENTS AND SUPPORTING MATERIALS	8
2.5 * DEFINITIONS AND ACRONYMS	9
3 * DESIGN CONSIDERATIONS	11
3.1 * REALISTIC CONSTRAINTS	11
3.2 SYSTEM ENVIRONMENT AND EXTERNAL INTERFACES	11
3.3 * INDUSTRY STANDARDS	11
3.4 * KNOWLEDGE AND SKILLS	13
3.5 * BUDGET AND COST ANALYSIS	14
3.6 * SAFETY	14
3.7 PERFORMANCE, SECURITY, QUALITY, RELIABILITY, AESTHETICS ETC.	15
3.8 * DOCUMENTATION	15
3.9 RISKS AND VOLATILE AREAS	15
4 * EXPERIMENT DESIGN AND FEASIBILITY STUDY	16
4.1 * EXPERIMENT DESIGN	16
4.2 * EXPERIMENT RESULTS, DATA ANALYSIS AND FEASIBILITY	17
5 * ARCHITECTURE AND HIGH LEVEL DESIGN	23
5.1 * SYSTEM ARCHITECTURE AND DESIGN	23
5.2 * HARDWARE ARCHITECTURE	24
5.3 * SOFTWARE ARCHITECTURE	26
5.4 * RATIONALE AND ALTERNATIVES	26
6 DATA STRUCTURES	28
6.1 INTERNAL SOFTWARE DATA STRUCTURE	28
6.2 GLOBAL DATA STRUCTURE	28
6.3 TEMPORARY DATA STRUCTURE	28
6.4 DATABASE DESCRIPTIONS	28

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

7 * LOW LEVEL DESIGN	29
7.1 * <i>RF20 LiDAR ODOMETRY GENERATOR</i>	29
7.2 * <i>PAN-TILT GIMBAL PROPORTIONAL CONTROLLER</i>	30
7.3 * <i>VISION-TRIGGERED AUTOMATED CAPTURE SYSTEM</i>	31
7.4 * <i>AUTONOMOUS EXPLORATION AND NAVIGATION SYSTEM</i>	32
7.5 * <i>LoRA C2 LINK</i>	34
7.6 * <i>PLANTCV ANALYSIS PIPELINE</i>	35
8 * TECHNICAL PROBLEM SOLVING	37
8.1 * ADDRESSING HOLONOMIC DRIVE SLIPPAGE AND ODOMETRY FAILURE	37
8.2 * SOLVING THE PROBLEM OF HOLONOMIC DRIVE SLIPPAGE AND ODOMETRY FAILURE	37
8.3 * MITIGATING EDGE-COMPUTING LATENCY AND GIMBAL JITTER	37
8.4 * SOLVING THE PROBLEM OF MITIGATING EDGE-COMPUTING LATENCY AND GIMBAL JITTER	37
8.5 * RESOLVING I2C COMMUNICATION LOCKUPS FROM MOTOR SURGES	37
8.6 * SOLVING THE PROBLEM OF I2C COMMUNICATION LOCKUPS FROM MOTOR SURGES	38
8.7 * NAVIGATING TIGHT AGRICULTURAL CORRIDORS AND COSTMAP INFLATION	38
8.8 * SOLVING THE PROBLEM OF NAVIGATING TIGHT CORRIDORS AND COSTMAP INFLATION	38
9 USER INTERFACE DESIGN	39
9.1 APPLICATION CONTROL	39
9.2 USER INTERFACE SCREENS	39
10 * TEST PLAN	41
10.1 * TEST DESIGN	41
10.2 * BUG TRACKING	42
10.3 * QUALITY CONTROL	42
10.4 * IDENTIFICATION OF CRITICAL COMPONENTS	42
10.5 * ITEMS NOT TESTED BY THE EXPERIMENTS	42
11 * TEST REPORT	43
11.1 * TEST 1: AUTONOMOUS VISION-TRIGGERED CAPTURE SYSTEM	43
11.1 * TEST 2: PLANT HEALTH DETECTION AND SEGMENTATION ACCURACY	43
11.1 * TEST 3: LoRA C2 LINK INPUT LATENCY	43
12 * CONCLUSION AND FUTURE WORK	44
12.1 * CONCLUSION	44
12.2 FUTURE WORK	45
12.3 * ACKNOWLEDGEMENT	46
13 * REFERENCES	47
14 * APPENDICES	48

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

1 * Executive Summary

The goal of this senior design project is to engineer an autonomous rover dedicated to monitoring plant and crop health with precision agriculture. In plants exerting signs of disease or nutrient deficiencies, our rover enables targeted intervention rather than indiscriminate application of treatments, which can lead to harmful chemical runoff and the waste of limited resources.

Health analysis of the flora is implemented using PlantCV, an open-source computer vision and machine learning (ML) pipeline. This analysis is conducted via a camera sensor rigged onto a pan/tilt armature on the rover to capture optimal images of the crops. Captured images are processed locally by the rover's onboard computer (Raspberry Pi). Data packets containing the diagnostic results of the post-processed images are then reliably transmitted to a base station system monitor via an onboard Long Range (LoRa) wireless antenna.

Implementation of autonomous navigation is conducted using a combination of 2D LiDAR sensor data and the ROS 2 navigation stack. The LiDAR sensor scans the perimeter of the rover, and this raw scan data is utilized by the onboard computer to construct a high-fidelity 2D map of the greenhouse or test environment using SLAM. Once built, this map functions as a “memory” to allow the rover to understand its surroundings. Using AMCL, the rover dynamically calculates its position on the map to autonomously navigate the area, guiding itself along rows of flora and avoiding obstacles while the vision system scans the plant beds.

To ensure operational safety and flexibility, the rover features a persistent manual control override. When the manual intervention signal (`CMD:MODE_MANUAL`) is received from the base station via the LoRa transceiver, the autonomous navigation algorithms are instantly bypassed, and direct control of the rover is securely handed over to a human operator via a manual controller.

The physical architecture of the rover consists of a 4WD differential-drive chassis, providing the necessary traction to traverse agricultural terrain, with the pan/tilt armature mounted securely on the body to allow the vision system a comprehensive range of vision.

2 * Introduction

2.1 * Design Objectives and System Overview

Our design concept is a hybrid remote-operated and autonomous rover capable of mapping, navigation, and real-time plant analysis through the implementation of LiDAR, motor control, computer vision, and long-range wireless communication. The primary objective is to advance the automation of crop maintenance by reducing the need for manual crop inspection in large agricultural fields. Our system collects accurate plant data and uses it to optimize resource allocation through precision agriculture. Guided by the research of intelligent systems, multiple technologies are incorporated into a complete and operational system.

Core subjects of electrical engineering displayed in our project include embedded systems, digital communication, sensor integration, and power electronics. These are reflected in practical implementations such as GPIO interfacing, UART serial communication, signal encoding, data acquisition, and power optimization. Our rover is intended for remote agricultural monitoring in isolated network environments where stable internet connection is limited. Utilization of long-range wireless communication supports precision plant analysis to inspect crops, detect disease, and track growth. Our system supports autonomous field mapping, used to generate optimized navigation paths for efficient coverage of crop fields and obstacle avoidance.

The user environment is managed from the base station, which features a keyboard and gamepad controller for manual rover navigation. Operation commands are sent from the base station to the rover through long-range wireless communication. The rover can switch to autonomous navigation through LiDAR-based scanning which develops an environmental map. Maneuvering through crop fields, the rover focuses on an orange flag through pan/tilt arm movements and captures an image of the plant using the front camera. This image is saved to a specific folder which enables plant health analysis to occur. Once this process has completed, the plant's overall health percentage is output and saved to the system.

Task Responsibilities:

Ying Gao: Responsible for the Software Architecture of the ROS 2 Navigation Stack, Vision Tracking Pipeline, and Gimbal Control. Specific quantitative objectives include:

Vision Processing Rate: Maintaining a real-time computer vision tracking loop at ≥ 30 Hz (latency ≤ 33 ms) on the edge device using HSV color space extraction. **Gimbal Stability:** Achieving a stable target lock with a strict 40-pixel dynamic deadband, ensuring 0 mechanical servo jitter or oscillation during active tracking. **Automated Capture Threshold:** Executing high-resolution image captures only after maintaining a continuous 3.0-second center-lock threshold to guarantee image clarity for PlantCV analysis. **Odometry Accuracy:** Generating reliable planar odometry via LiDAR-based RF2O with an update rate of ≥ 10 Hz, successfully compensating for physical Mecanum wheel slippage to maintain translational mapping errors < 0.15 meters during autonomous Nav2 exploration.

Angel Pulido: Responsible for managing the latency in the LoRa C2 link during manual control to be within a < 50 millisecond range. This will ensure that the latency is imperceptible to the human eye resulting in fluid controller input to rover motion response.

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

Adam Lachman: Responsible for PlantCV processing ≤ 10 seconds per image, plant health percentage performing within 30% of expected value, feature extraction consistency $\geq 90\%$ across images of the same plant, strength against lighting variation maintained $\geq 80\%$ across $\pm 20\%$ illumination changes

2.2 * Backgrounds and Prior Art

A review of existing literature shows that agricultural rovers and intelligent crop monitoring systems are an active and rapidly evolving area of research. One example is the Mineral project developed by Alphabet, which incorporates cameras, sensors, and GPS to capture high-resolution images of each individual plant in the field. These systems integrate soil and weather data to enable predictive modeling and yield estimation of entire crop fields. Another existing project is the Agrova rover which utilizes edge AI for real-time detection and complex disease classification. Their models have achieved high accuracy using deep learning models operating on embedded hardware. Other rover designs in academic projects implement multifunctionality such as crop harvesting, soil nutrient testing, and environment specialization.

Our agricultural rover emphasizes a balanced and cost-effective design accessible for small scale deployments. Compared to large-scale industry solutions, our rover achieves autonomous operation intended for strict environments. Other systems are typically limited to specific environments that rely on extensive infrastructure and focus on single-purpose tasks. Implementation of multiple functionalities such as LiDAR-based mapping and computer vision convey the use of our rover as a full-fledged system. Some shortcomings of our rover compared to advanced commercial systems included lower data throughput, reduced scalability, and less sophisticated machine learning models. Our system is still well-suited for real-world precision agriculture application in resource-constrained settings.

2.3 * Development Environment and Tools

The hardware development environment includes a Raspberry Pi running Linux as the main processing unit for LiDAR-based navigation, plant analysis, camera operation, and motor control. Two ESP32 microcontrollers are utilized for long-range wireless communication between the base station and the rover. The system is powered by a 3S LiPo battery with voltage regulation to supply stable power to all components. For software tools, Python is used to implement PlantCV analysis while other libraries such as OpenCV and NumPy support image processing. YOLO is used to regulate image capture while ROS2 and FoxGlove software are used to assemble and visualize LiDAR scanning data. Testing equipment includes a laptop for the base station, a controller for manual rover control, and manufactured plants for health analysis.

2.4 * Related Documents and Supporting Materials

The rover's communication architecture relies on several internationally recognized standards to ensure interoperability. Real-time telemetry is streamed via the IETF RFC 6455 (WebSocket) protocol, while high-level robotic messaging is governed by the OMG DDS standard. Physical sensor interfacing is conducted through the TIA/EIA-232 (UART) serial standard. Coordinate frames ([map](#), [odom](#), [base_link](#)) are tethered and named in compliance with ROS REP 105 standards to ensure the rover can interface with various third-party LiDAR or SLAM software.

Imaging and navigation systems utilize common image and data standards. Outputs from the PlantCV image analysis rely on the ISO/IEC 10918-1 (JPEG) digital compression and coding standards for the

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

continuous-tone still images produced from the machine learning algorithms. The transparent map layers in SLAM are composed using ISO/IEC 15948 (PNG) standard raster graphics.

The Raspberry Pi 5 is host to a suite of communication and networking protocols to establish a secure link between hardware and software components. Broadband data transmission between the RPi5 and base station are carried out over IEEE 802.11 (Wi-Fi) standards. Narrowband and C2 links to the base station are handled by the LoRa/LoRaWAN v1.0.4 radio communication standard.

2.5 * Definitions and Acronyms

Table 2.1 Definitions and Acronyms

Acronym	Definition	Acronym	Definition
RPi5	Raspberry Pi 5	$n_{preamble}$	Preamble
FSPL	Free Space Path Loss	$T_{preamble}$	Preamble Duration
R_b	Data Rate	PL	Payload Size [bytes]
T_{packet}	Total Time-On-Air	H	Header
d	distance	DE	Low Data Rate Optimization
SF	Spread Factor	$T_{payload}$	ToA of Physical Payload Portion
BW	Bandwidth	FEC	Forward Error Correction
CR	Coding Rate	ToA	Time-On-Air
CSS	Chirp Spread Spectrum	QoS	Quality of Service
PWM	Pulse Width Modulation	DDS	Data Distribution Service
AMCL	Adaptive Monte Carlo Localization	SLAM	Simultaneous Localization and Mapping
f	Frequency	C2	Command and Control

Team 24-Sec.022 <u>Dept. of Electrical and Computer Engineering, UCR</u>	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

Table 2.1 Definitions and Acronyms

GPS	Global Positioning System		
-----	---------------------------	--	--

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

3 * Design Considerations

3.1 * Realistic Constraints

Most constraints we encountered came as a result of the economical budget placed on this project. This restricted our selection of advanced hardware as we strived to maintain adequate performance while utilizing cost-effective components. With the chassis selected, we faced size and mobility constraints due to the compact rover platform. This affected component placement and reduced our ability to implement further features. Various component constraints include limited LiPo battery capacity, reduced memory and CPU allocation for Raspberry Pi processes, minimal range for LiDAR scanning, and lower resolution for camera image capture. Software constraints stem from use of open-source frameworks which reduces ability for customization and development. Time constraints due to the academic quarter restricted our ability to conduct in-depth tests on our system that could enable overall system improvements.

3.2 System Environment and External Interfaces

The system consists of a mobile rover platform and a remote base station, communicating via ESP32 microcontrollers utilizing LoRa wireless communication. On the rover, a Raspberry Pi 5 acts as the central processing unit for LiDAR, motor, and camera operations. These components are controlled by GPIO and PWM pin connections, as well as a MIPI CSI-2 connector. UART serial communication is utilized between the onboard Raspberry Pi 5 and ESP32, as well as gamepad controller and keyboard input at the base station. Voltage levels are regulated from the 12.6VDC 3S LiPO battery through a voltage divider leading 2.1V to the ESP32 and a buck converter leading 5V to the Raspberry Pi 5.

3.3 * Industry Standards

Table 3.1 System Compliance with Industry Standards

Standard (Reference)	Technology (Component)	Compliance Strategy	Design Impact
TIA/EIA-232-F (Serial Communication)	UART (LD19 LiDAR & LoRa modules)	Hardware wired to RPi5 GPIO RX/TX pins. Software utilizes POSIX device drivers.	Requires matching 3.3V logic levels to avoid frying the Pi. LiDAR requires a 230400 baud rate while LoRa was set to 115200 baud.
IEEE 802.11ac (Wireless LAN)	Wi-Fi (Broadband Telemetry & PlantCV)	The Pi uses its WLAN interface to connect to a local router/hotspot using standard WPA2-PSK security.	Range limitation facilitated the LoRa communication system.
LoRaWAN v1.0.4	LoRa	Transceivers configured	Forced system onto 915

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

Table 3.1 System Compliance with Industry Standards

(Semtech/LoRa Alliance)	(Narrowband Telemetry / Manual Control)	to use CSS modulation for C2 link.	MHz ISM band for US compliance. Strict BW limitations enforced a maximum packet size of 255 bytes.
OMG DDS v1.4 (Data Distribution Service)	ROS 2 Middleware (Node Communication)	Eclipse Cyclone DDS utilized for peer-to-peer networking.	Required tuning of QoS parameters-setting LiDAR topic reliability to “best_effort” and a queue size of 10 to prevent buffer overflow and memory crashes.
ROS REP 105 (ROS Enhancement Proposal)	Coordinate Frames (Navigation & AMCL)	Static transform publishers were coded to link hardware to digital map.	Dictated the software architecture. We were forced to map physical offsets in meters and name frames strictly as <code>map→odom→base_link→base_laser</code>
IEEE 1003.1 (POSIX)	Operating System (Rover Core)	The system runs on Ubuntu Linux (POSIX compliant).	The POSIX standard meant all hardware had to be accessed as file streams.
IETF RFC 6455 (WebSocket Protocol)	Live UI & Telemetry (Foxglove Bridge)	The <code>foxglove_bridge</code> ROS 2 package spawns a WebSocket server to stream real-time robotic data directly to the base station.	Bound to TCP Port 8765. Allowed for the low latency needed to stream 2D LiDAR point clouds at a continuous 20 Hz.
ISO/IEC 10918-1 (JPEG Image Compression)	Machine Vision (PlantCV Camera)	The PlantCV pipeline captures raw RGB arrays from the camera and applies JPEG compression before saving diagnostics to the SD card.	Reduced file sizes of 1080p camera captures from several megabytes of raw data to ~200-500 KB per image.
ISO/IEC 15948	Map Generation	The	Allowed for strict

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

Table 3.1 System Compliance with Industry Standards

(PNG - Portable Network Graphics)	(SLAM / Map Saver)	<code>nav2_map_server</code> exports 2D occupancy grids as a lossless PNG image paired with a YAML metadata file.	lossless encoding. Pixel values can be precisely mapped to physical space-a grayscale value corresponds to 0.05 meters per pixel
-----------------------------------	--------------------	---	--

3.4 * Knowledge and Skills

Engineering this autonomous agricultural rover required a multidisciplinary synthesis of robotics, software engineering, embedded hardware design, and network administration. At the system's core, the project demanded proficiency in the Robot Operating System 2 (ROS 2) and its DDS middleware to implement complex coordinate frames and navigation algorithms, including SLAM and AMCL. Developing this autonomy stack required advanced software engineering skills across multiple languages: C++ for compiling high-performance hardware nodes, Python for writing the mission commander and PlantCV machine learning pipelines, and Linux Bash scripting for automated process management on a headless operating system. Bridging the software and physical areas required embedded systems expertise to interface the Raspberry Pi with external components, utilizing UART serial communication for the LiDAR and LoRa transceivers, alongside GPIO and PWM for precise motor and gimbal control. Finally, the rover's remote capabilities necessitated strong networking and telecommunications knowledge to reliably transmit real-time telemetry and compressed crop-health data using a combination of LoRa RF modulation, WebSockets, and HTTP protocols.

Angel Pulido: Courses I have taken that are related to this project include

• EE 142: Introduction to Machine Learning and Data Mining • EE 115: Introduction to Communication Systems • EE 110AB: Signals and Systems • EE 141: Digital Signal Processing • EE 132: Automatic Control • EE 175AB: Senior Design	• EE 152: Image Processing • EE 146: Computer Vision • EE 30AB: Fundamentals of Electric Circuits • EE 120AB: Logic Design / Introduction to Embedded Systems in C • EE 128: Sensing and Actuation for Embedded Systems
New Knowledge	
• Robotic Middleware: ROS 2 & DDS ◦ Publish/Subscribe mechanics • Probabilistic Robotics: ◦ SLAM and AMCL ◦ Transform Trees	• RF Telemetry: LoRa ◦ CSS modulation ◦ LoRa bandwidth optimization • System Engineering: ◦ Edge Computing

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

Ying Gao: My academic background in Precision Instrumentation, along with my coursework at UC Riverside—specifically **EE268: Trustworthy AI for Autonomous Systems** and courses on **Real-Time Embedded Systems**—provided the foundational theory for this project. Developing the autonomous navigation and targeting systems required a deep synthesis of control systems, computer vision, and ROS 2 middleware. To overcome the lack of physical wheel encoders and the inherent slippage of our Mecanum wheels, I independently researched and implemented RF2O (Range Flow-based 2D Odometry) to extract planar odometry directly from LiDAR scans. Additionally, I learned to structure the environment using fiducial markers (orange flags) and developed a custom OpenCV HSV color-tracking pipeline. This required applying Proportional-control (P-control) theory, which I learned from my coursework, to design a dynamic deadband algorithm that successfully eliminated mechanical servo jitter in the pan-tilt gimbal.

Adam Lachman: Prior coursework that assisted in the implementation of PlantCV analysis include EE016: Data Analysis in Engineering Applications, EE020B: Linear Methods for Engineering Analysis & Design using MATLAB, and EE142: Intro to Machine Learning and Data Mining. These courses provided a strong theoretical and practical foundation in algorithm development and image processing necessary for plant health analysis. Experience with Python programming greatly supported the implementation and debugging process of PlantCV. Concepts incorporating MATLAB and machine learning helped strengthen familiarity with supervised learning, classification methods, and model evaluation techniques. New knowledge necessary for completing this project included research of PlantCV documentation, managing conda environments for testing models, and utilizing Raspberry Pi 5. Understanding other aspects of our project, including system design and hardware assembly, were supported by courses such as CS120B: Introduction to Embedded Systems, EE100B: Electronic Circuits II, and EE128: Sensing and Actuation for Embedded Systems. These courses provided foundational knowledge of communication protocols, firmware development, and sensor integration through microcontroller interfacing.

3.5 * Budget and Cost Analysis

For our rover, the MC100 4WD Mecanum Wheel Car Chassis was selected at a price of \$29.99. It was our lowest cost option that maintained enough platform space to fit other components. Two TB6612FNG DC Motor Drive Modules were required to be purchased at a cost of \$5.99. We selected the OVONIC 3S LiPo battery at \$26.99. It was our most expensive option but seemed necessary to sufficiently power the rover for long periods of time. For image capture, we acquired the Arducam 5MP Camera at \$6.99 and the corresponding pan tilt platform at \$26.99. We also purchased the FHL-LD19 2D LiDAR Sensor at \$69.99, as it provided increased functionality compared to cheaper options. Components at hand used include the Raspberry Pi 5 at ~\$95 and two ESP32 LoRA V3 at ~\$35 each. Totaling all components together, our budget sums up to approximately \$332.00.

3.6 * Safety

Safety considerations for our rover include electrical components, mechanical functions, communication procedures, and navigation hazards. Our primary safety objective is to ensure all circuits operate within safe voltage/current limits. Moving parts including the wheels and pan tilt arm must be secured onto the rover platform to reduce risk of damage. Smooth operation of LoRa communication is necessary to control rover navigation and relay important information to and from the base station. Navigation hazards will be avoided through the use of LiDAR scanning technology.

Team 24-Sec.022 <u>Dept. of Electrical and Computer Engineering, UCR</u>	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

3.7 Performance, Security, Quality, Reliability, Aesthetics etc.

N/A

3.8 * Documentation

Technical documentation is created alongside system development to ensure traceability, clarification, and consistency through the implementation process. For any procedures that require a lengthy list of steps, instruction documents were produced and shared with other team members. These documents are kept up to date during development and experimentation phases of our project. Version control procedures are implemented using a repository-based system where all code and documentation can be stored.

3.9 Risks and Volatile Areas

N/A

4 * Experiment Design and Feasibility Study**4.1 * Experiment Design****Experiment I: Drone versus Rover - Angel Pulido**

We considered whether we would like to design an Unmanned Autonomous Vehicle (UAV) multicopter or a rover ground vehicle. Ultimately we settled on designing a rover. Although the main point of concern was the affordability of each vehicle type, studying their pros and cons reveals a more divisive study:

1. For our use-case, close-range analysis is needed to adequately capture an image of a leaf. A rover will be much more stable; ideally eliminating motion blur caused by the instability of a drone fighting winds and making micro-corrections.
2. The net payload's main contributions include a LiDAR, a Raspberry Pi 5, the camera armature, and a 12 V Lithium Ion battery. The drone requires delicate analysis of the net weight and motor torque. Because the drone is required to lift off the ground it has a lower payload capacity. Whereas the rover can easily carry each component.
3. Navigation is limited to 2-D movement with the rover. Considering the precision needed to analyze a plant leaf, this movement is sufficient and offers greater control. A drone would be navigating a 3-D environment. This would introduce unstable image captures due to the micro-corrections.
4. A drone would have more mobility than a rover as it could simply fly over ground obstacles to reach its destination. In contrast a rover is heavily limited by terrain such as rocks, mud, and dense grass.
5. The Field of View (FOV) of a drone allows it to quickly scan an entire field from an aerial view. Whereas a rover would only have a ground view, barring it from top-down views that would be needed for trees.

Experiment II: Battery Size/Capacity - Angel Pulido

Due to the size of the rover chassis, careful selection of the battery is needed to account for the available surface area. The metal panel of the chassis has a width of 75 mm and a length of 135 mm. This means system components must be carefully placed so as to fit together without interfering with one another.

Experiment III: Power Draw and Runtime - Angel Pulido

Derive a total runtime for the rover based on the power draw of each hardware component. Components will be systematically measured for their actual current and power consumption.

Experiment IV: Power On Test - Angel Pulido

Determine if the rover's voltage levels at the buck converter rail, motor driver rail, and voltage divider are adequately within their expected voltage range. Using a multimeter, the voltage at each output of the battery was measured while connected to power.

Experiment V: LoRa C2 Link / UART Optimization - Angel Pulido

Fine-tune the LoRaWAN communication protocol to eliminate latency between controller input and wheel movement as well. Also modify the `check_serial` function in the `rover_core.py` script to stop latency. LoRa tuning will be performed via quantitative analysis while the UART tuning will be done programmatically.

Experiment VI: Vision Targeting Deadband Optimization - Ying Gao

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

Objective: Determine the optimal pixel deadband threshold for the pan-tilt gimbal P-controller to eliminate mechanical servo jitter without sacrificing target tracking responsiveness. Setup & Procedure: The rover was placed at varying distances (1m to 5m) from an orange fiducial marker. The P-control algorithm was tested with deadband values ranging from 0 to 80 pixels. System stability (oscillation frequency) and response latency were recorded via ROS 2 /vision/status topic echoes.

4.2 * Experiment Results, Data Analysis and Feasibility

Experiment I Results - *Angel Pulido*

- Although the drone allows for a greater FOV and mobility, a rover has a stability that is crucial for the study of chlorosis in plant leaves.

Experiment II Results - *Angel Pulido*

After researching several battery capacities, the following were found:

Table 4.1 Battery Size/Capacity Comparison

#	Capacity	Weight	Dimensions	Viability	Pass
1	6700 mAH	486 g	154.7 mm × 46.9 mm × 30 mm	Large / Heavy	✗
2	5200 mAH	358 g	135 mm × 42 mm × 28 mm	Large / Heavy	✗
3	2200 mAH	179 g	105 mm × 33 mm × 20 mm	Long	✗
4	1300 mAH	116 g	75 mm × 36 mm × 22 mm	Adequate Size / Small Capacity	✓
5	2200 mAH	137 g	75 mm × 34 mm × 26.5 mm	Adequate Size / Capacity	✓
6	2200 mAH	240 g	75 mm × 33 mm × 25 mm	Adequate Size / Capacity	✓

Larger battery capacities directly correlate to longer operational times. Although the ideal battery would carry a surplus of milliampere-hour (maH), its weight and surface area must be taken into consideration. For this reason, among the batteries in the above table, the only practical batteries will be 4-6. Batteries 5 and 6 have an efficient capacity and dimension. Among these two batteries, we decided to utilize battery 5 because it is lighter and was slightly more inexpensive.

Experiment III Results - *Angel Pulido*

Available Energy

To determine the runtime of the system, the usable energy in Watt-Hours (Wh) must first be determined.

- Battery:** 11.1V 3S LiPo, 2200mAh (2.2Ah)**Total Energy (Wh):**

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

$$11.1\text{ V} \times 2.2\text{ Ah} = 24.42\text{ Wh} \quad (4.1)$$

- **Usable Energy** (Above 20% capacity threshold):

$$24.42\text{ Wh} \times 0.8 = 19.5\text{ Wh (Usable)} \quad (4.2)$$

Breakdown

Table 4.2A 5V Logic Rail (Powered by Buck Converter)

Component	Estimated Current @ 5V	Power [Watts]	Notes
Raspberry Pi 5	~1.8A (Avg Active)	9.0W	High load from SLAM (LiDAR) & Video processing
FHL-LD19 LiDAR	~0.2A	1.0W	Always on (scanning)
OV5647 5MP Camera Module	~0.25A	1.25W	Capturing Images & Video
Heltec V3	~0.15A	0.75W	Intermittent transmission spikes
Pan-Tilt Servos	~0.2A (Avg)	1.0W	Constant movement (scanning)
Buck Efficiency Loss	N/A	~1.5W	~10% energy lost as heat in conversion
Rail A Total	~2.6A	~14.5W	Constant drain while rover is ON

Table 4.2B 11.1V Motor Driver Rail

Component	Estimated Current @ 5V	Power [Watts]	Notes
TT DC Motors ×4	~1.2A (Total)	~13.5W	Assuming a 6V average (PWM) under load
L298N Heat Loss	N/A	~3.0W	Drops ~2V as heat
Rail B Total	~1.2A @ 11V	~16.5W	Only during movement

Total System Power & Runtime

- **Scenario A:** Heavy Use (Full Speed + Processing)

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

Always driving, running YOLO, SLAM, PlantCV, and moving servos.

- Total Power: 14.5W (Logic) + 16.5W (Motors) = 31W
- Estimated Runtime:

$$19.5 \text{ Wh (Usable)} \div 31 \text{ W} \approx \mathbf{37 \text{ minutes}} \quad (4.3)$$

- **Scenario B:** Mixed Use (Stop-and-Go)

Driving 50% of the time, stopping to look around, idling to process.

- **Average Power:** 14.5W (Logic) + 8.25W (Motors @ 50%) = 22.75W
- **Estimated Runtime:**

$$19.5 \text{ Wh (Usable)} \div 22.75 \text{ W} \approx \mathbf{51 \text{ minutes}} \quad (4.4)$$

These results show that a more methodical approach allows for a longer operation time - **Scenario B**.

Experiment IV Results - Angel

Table 4.3 Power On Test Results

Component	Expected Voltage [V]	Actual Voltage [V]
Rail A (RPi 5 Rail)	5	~5.1
Rail B (L298N Rail)	12.6	~11.1
Voltage Divider Output	2.1	~1.9

These results demonstrate that the voltages are within adequate ranges from their expected values. The test was performed while the battery was at a full charge (~11.1V).

Experiment V Results - Angel

UART Parallel Bus

Both the USB connection from the local LoRa module connected to the base station and the remote module connected to the Pi use a baud rate of 115200 to ensure the rover can stream command and sensor data at a high-speed. To reduce lag time over long operation times, the `check_serial` function was modified to process every command instantly instead of individually.

```
def check_serial(self):
    """ Reads data without throwing away packets """
    if self.ser is None:
        return
```

```
# FIX 3: Process EVERY command instantly to stop latency
while self.ser.in_waiting > 0:
    try:
        line = self.ser.readline().decode('utf-8').strip()
        if line:
            self.procCmd(line)
    except Exception:
        pass
```

LoRaWAN Communication

Three important factors of the P2P LoRa connection are Free Space Path Loss, Data Rate, and time-on-air. FSPL is the decrease in signal strength of a signal traveling between two antennas on a line-of-sight path through free space. It can be determined with the formula

$$FSPL = 20 \log_{10}(d) + 20 \log_{10}(f) - 147.55 \quad (4.5)$$

If the rover will, at maximum, be a distance of 12.9m (the approximate end-to-end distance in a large room) and the frequency is at 915MHz, the FSPL is then computed as

$$FSPL = 20 \log_{10}(12.9) + 20 \log_{10}(915) - 147.55 \approx 53.89 \text{ dB} \quad (4.6)$$

Because LoRa modules typically have a transit power of +14dBm and a sensitivity at SF7/BW500 of about -119dBm, a 54dB loss is negligible.

The LoRaWAN data rate is the speed at which data is transmitted from one point to another via a wired or wireless medium. It can be determined with the formula

$$R_b = SF \cdot \frac{4/(4+CR)}{T_{sym}} \cdot 1000 = SF \cdot \frac{4/(4+CR)}{2^{SF/BW}} \cdot 1000 \quad (4.7)$$

where SF is the spreading factor, CR is the coding rate, and BW is the bandwidth.

SF determines the number of chirps used to represent a single piece of data and essentially controls the tradeoff between range and speed. LoRa uses Chirp Spread Spectrum (CSS); where a chirp is a signal that increases or decreases in frequency over time. Spreading factor (typically ranging from 7 to 12) represents the power of two chirps:

- **High SF (e.g. SF12):** Uses more chirps which allows a packet to travel farther at the cost of speed.
- **Low SF (e.g. SF7):** Uses fewer chips. Making the transmission fast but limited to short distances.

The coding rate factor refers to the Forward Error Correction (FEC) used to protect the data stream

against interference and noise. Represented as a fraction $\frac{4}{4+CR}$, high and low values of CR correspond to

- **4/5(CR = 1):** Low overhead, higher data rate, but more susceptible to noise.
- **4/8(CR = 4):** High overhead, lower data rate, but more robust towards noise.

R_b can be optimized for a variety of distances by plotting the data rate as a function of spreading factor for chosen bandwidths as shown in **Fig. 1** (assuming $CR = 1$). From the figure, a bandwidth of 500kHz at $SF = 7$ has the highest data rate at the cost of range.

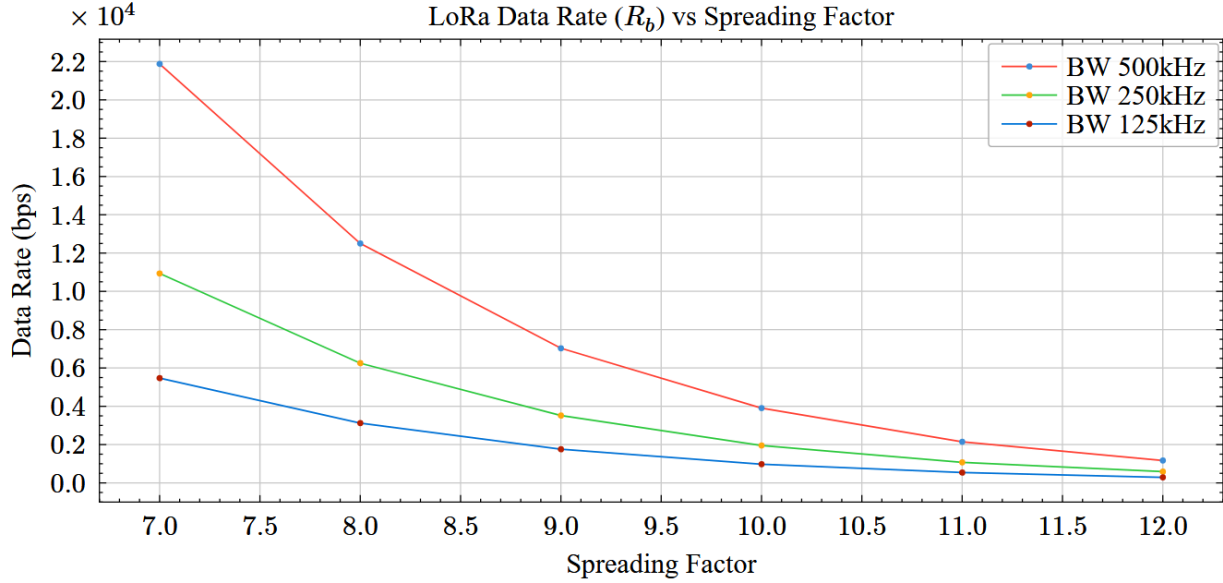


Figure 1: Plot of values of R_b for various values of BW and SF.

In addition, the time-on-air or airtime is the duration a LoRa packet takes to travel between modules. It can be found with the equation

$$T_{packet} = T_{preamble} + T_{payload} \quad (4.8)$$

where

$$T_{preamble} = T_s(n_{preamble} + 4.25) \quad (4.9)$$

and

$$T_{payload} = T_s \left[8 + \max\left(\text{ceil}\left(\frac{8PL - 4SF + 28 + 16 - 20H}{4(SF - 2DE)}\right)(CR + 4), 0\right) \right] \quad (4.10)$$

The preamble duration $T_{preamble}$ can be computed by first specifying the number of symbols in the preamble $n_{preamble}$; in the `unofficial_heltec.h` library this is set to 8 by default to ensure the device is interoperable between different devices and gateways. **Eq. 4.9** then becomes

$$T_{preamble} = T_s(8 + 4.25) = T_s(12.25) \quad (4.11)$$

here symbol transmission time $T_s = \frac{2^{SF}}{BW}$.

The payload and header duration $T_{payload}$ depends on several factors, namely the payload size in bytes PL, the header H, low data rate optimize DE. During manual navigation, commands are sent as string data in the form `M:0.00,0.00`. This string contains a 13-byte payload, therefore $PL = 13$. In order to include CR and PL in data packets, an explicit header must be use, that is $H = 0$. Lastly, DE is a parameter that

must be enabled if the symbol transmission time takes too long to send (usually for large data packets); DE is therefore set to 0 (disabled) since transmitted packets are small. The payload duration for the rover's specific configuration then becomes

$$\begin{aligned} T_{\text{payload}} &= T_s \left[8 + \text{ceil}\left(\frac{8(13)-4SF+28+16-20(0)}{4(SF-2(0))}\right) \cdot (1 + 4) \right] \\ &= T_s \left[8 + \text{ceil}\left(\frac{148-4SF}{4SF}\right) \cdot 5 \right] \end{aligned} \quad (4.12)$$

Therefore, combining Eqs. 11 and 12, Eq. 4.8 expands to

$$\begin{aligned} T_{\text{packet}} &= 12.25 \cdot \frac{2^{SF}}{BW} + \frac{2^{SF}}{BW} \cdot \left[8 + \text{ceil}\left(\frac{148-4SF}{4SF}\right) \cdot 5 \right] \\ &= \frac{2^{SF}}{BW} \cdot (12.25 + 5 \cdot \text{ceil}\left(\frac{37}{SF}\right) + 3) \\ &= \frac{2^{SF}}{BW} \cdot (15.25 + 5 \cdot \text{ceil}\left(\frac{37}{SF}\right)) \end{aligned} \quad (4.13)$$

Fig. 2 shows the time-on-air plotted as a function of the spreading factor for several bandwidths. This plot verifies the theoretical results seen earlier where lower bandwidths and spreading factors invite more latency.

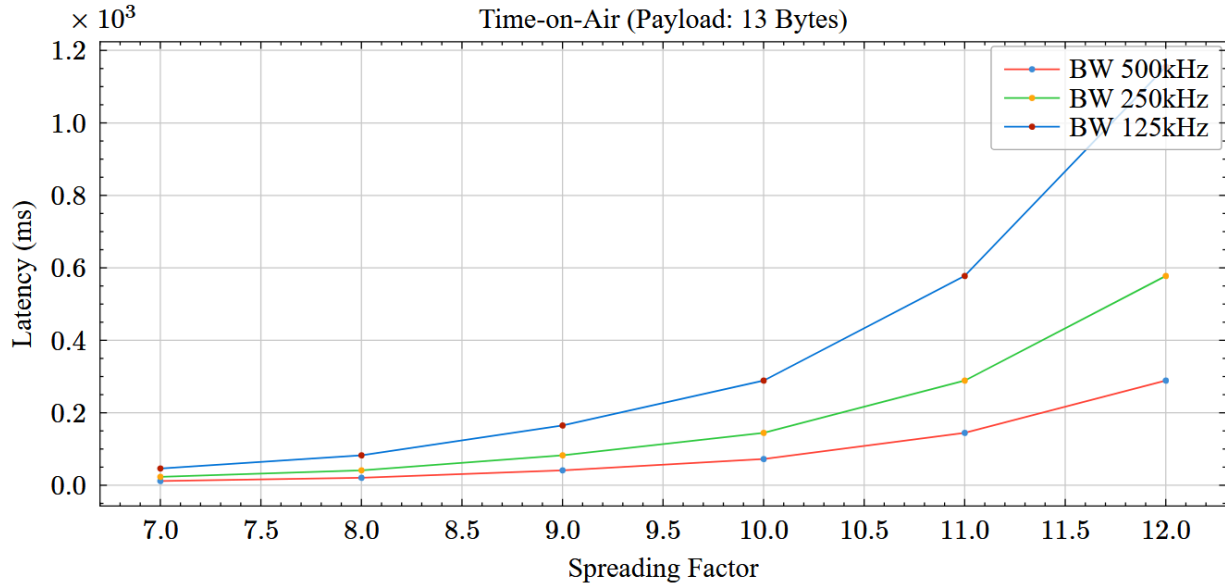


Figure 2: Plot of values of P_{payload} for various values of BW and SF.

These results demonstrate that for our current use-case, a bandwidth of 500kHz and spreading factor of 7 will give the least latency. The drawback is that a higher bandwidth also results in higher power draw.

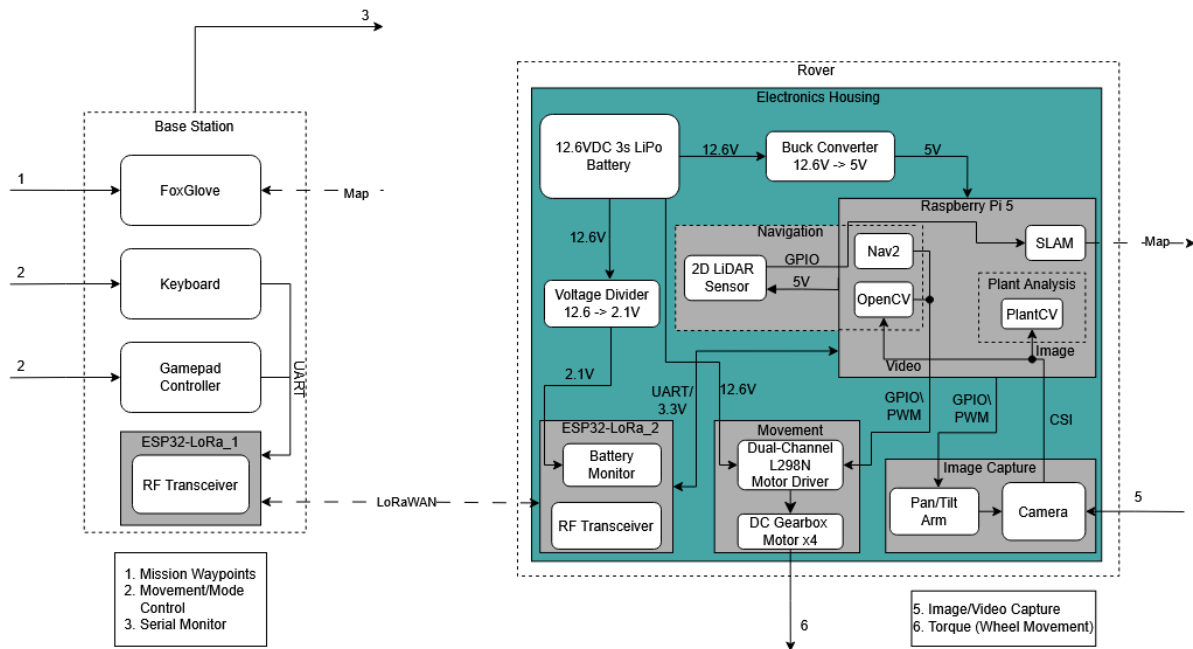
Experiment VI Results Ying Gao

The data indicated that a deadband of 0-20 pixels caused the servos to continuously hunt for the exact center, resulting in violent high-frequency jitter. Deadbands above 60 pixels caused unacceptable tracking lag, where the target could almost leave the camera frame before actuation occurred. A deadband of exactly 40 pixels proved to be the optimal mathematical threshold, perfectly eliminating all mechanical jitter while keeping the target securely centered for the PlantCV capture trigger.

5 * Architecture and High Level Design

5.1 * System Architecture and Design

Figure 5.1 System Block Diagram



At a high level, the system is composed of the base station and the rover platform. The base station is utilized for initial setup, manual control, and data visualization. It functions as the origin for movement commands that are sent to the rover through LoRa wireless communication. On our rover platform, information is received and transmitted to other various subsystems. Our Raspberry Pi 5 is the main component that manages motor control, pan tilt arm movement, image capture, plant analysis, and LiDAR sensing. It is powered by a chargeable battery with voltage regulation components.

Task Responsibilities:

Ying Gao: Responsible for the Software Architecture of the ROS 2 Navigation Stack (Nav2/SLAM/RF2O), the Vision Tracking Pipeline (OpenCV), and the Hardware Architecture for the Pan-Tilt Gimbal P-Controller.

Angel Pulido: Responsible for the LoRa C2 link and telemetry, the rover hardware design and construction, and the Foxglove bridge for SLAM visualization.

Adam Lachman: Responsible for the development, testing, and implementation of PlantCV software for plant health analysis along with the construction of plant models for live demonstrations.

5.2 * Hardware Architecture

Figure 5.2.1 Hardware Block Diagram

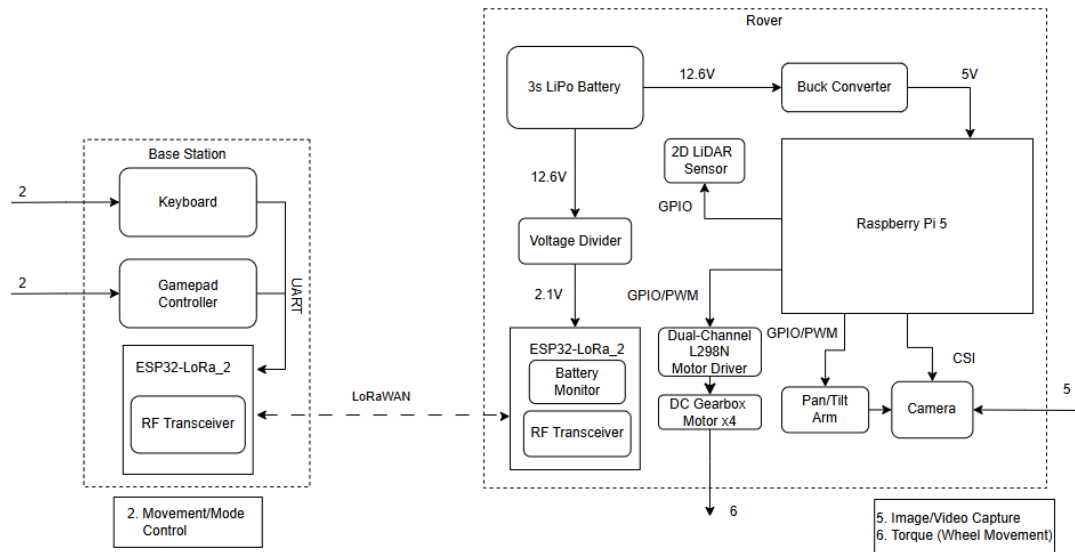


Figure 5.2.2 Rover Front

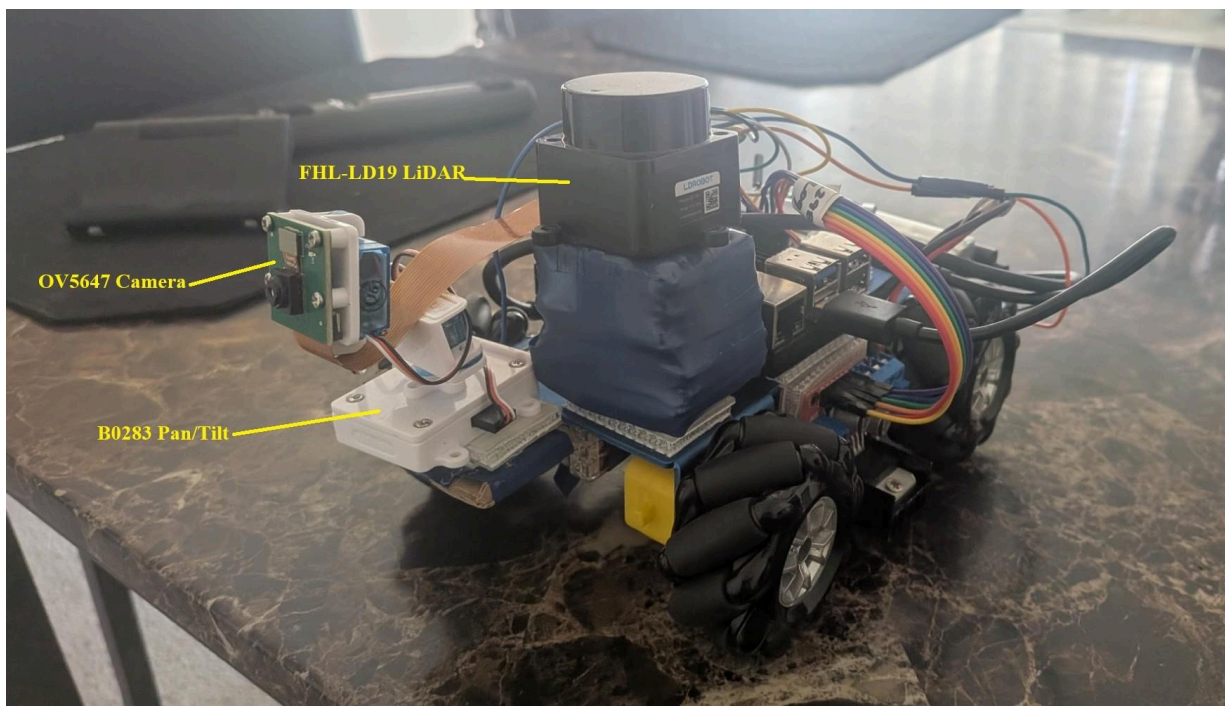


Figure 5.2.3 Rover Side

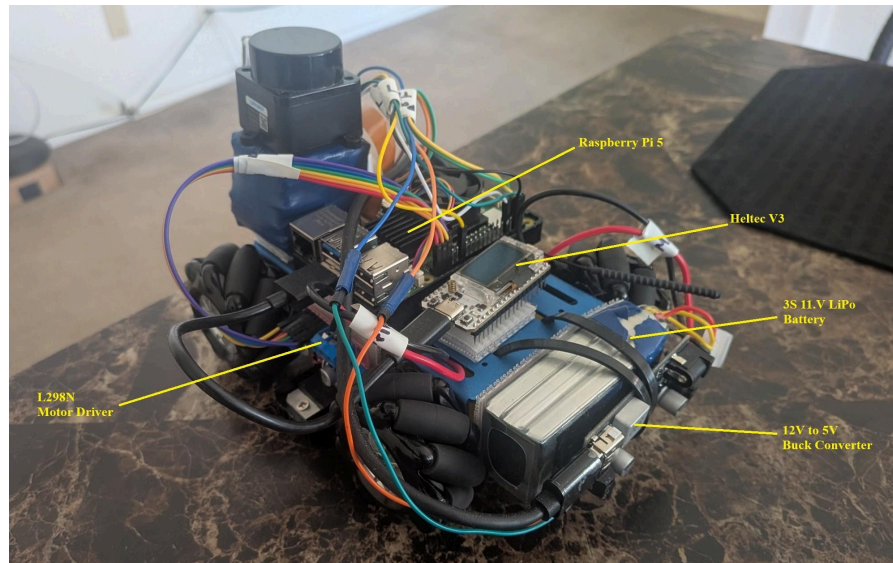
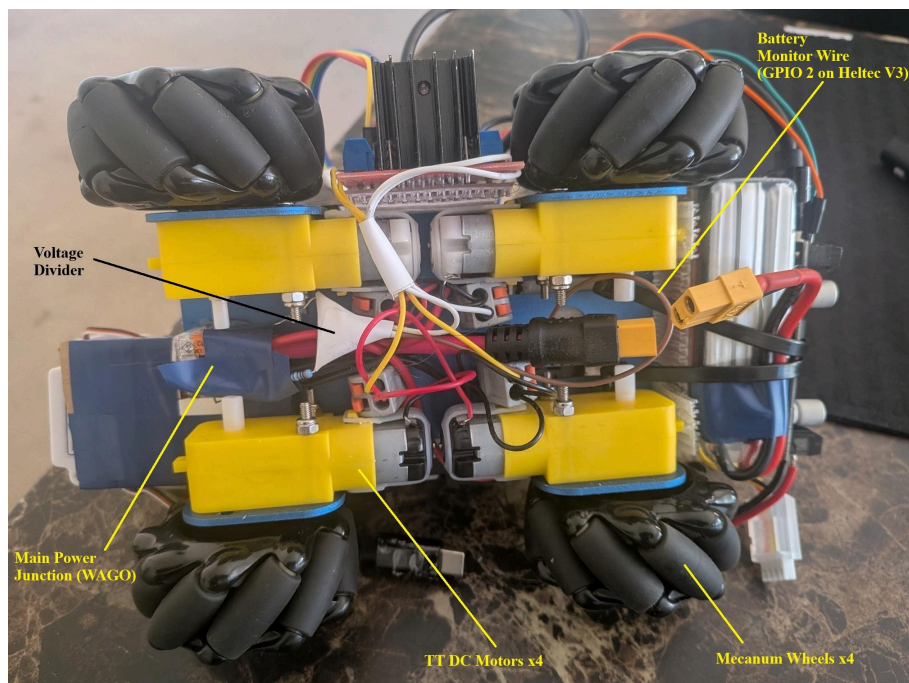


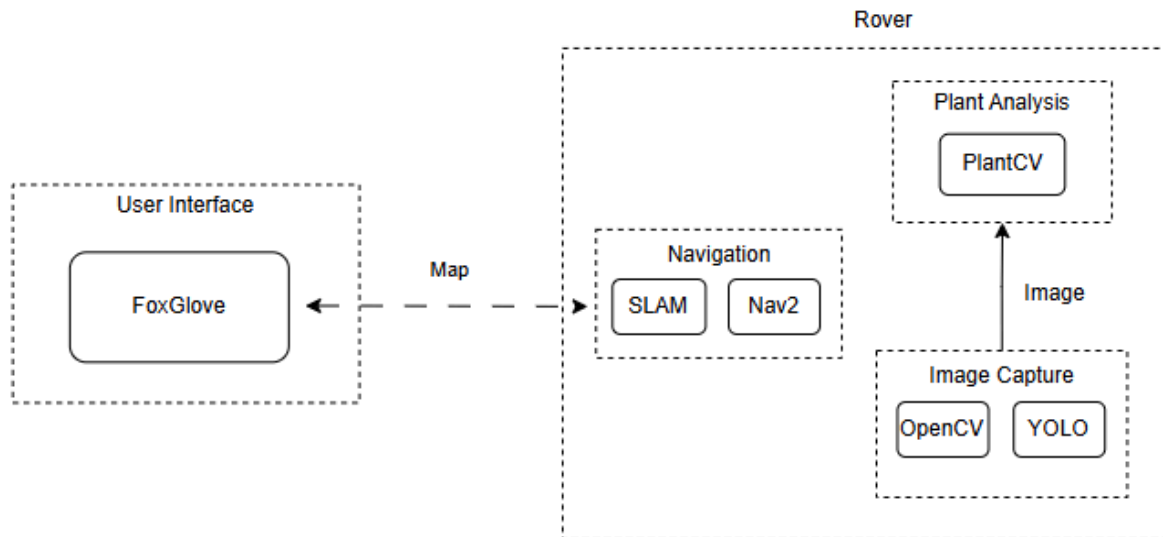
Figure 5.2.4 Rover Underside



Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

5.3 * Software Architecture

Figure 5.3.1 Software Block Diagram



5.4 * Rationale and Alternatives

Alternative: We initially considered using YOLOv8 for direct plant disease detection and navigation.

Rationale for Current Approach: YOLO inference on the Raspberry Pi 5 caused severe computational bottlenecks, dropping the frame rate and causing the pan-tilt gimbal to oscillate violently. We pivoted to "Structuring the Environment." By using an orange flag as a fiducial marker, we switched to a computationally inexpensive OpenCV HSV tracking pipeline. This freed up the CPU, achieved a stable 30Hz control loop for the motors, and allowed us to run the heavy PlantCV analysis only on static, locked-on snapshots. During the architectural design phase, the team initially considered deploying a heavy deep-learning object detection model (such as YOLOv8) for direct plant disease recognition and navigation targeting. However, preliminary testing revealed that running YOLO inference on an edge device like the Raspberry Pi 5 caused a severe computational bottleneck. The resulting low frame rate introduced significant latency into the control loop, causing the pan-tilt gimbal to oscillate violently and lose track of moving targets.

To resolve this, we adopted a classic engineering approach known as "Structuring the Environment." Instead of forcing a heavy neural network to interpret unstructured natural surroundings in real-time, we introduced highly visible fiducial markers (orange flags) to designate target plants. This rationale allowed us to replace the YOLO model with a computationally lightweight OpenCV HSV color-tracking pipeline. This alternative easily maintains a stable 30Hz control loop, ensuring smooth and responsive servo tracking. By utilizing this alternative, we successfully reserved the heavy, processor-intensive image

Team 24-Sec.022 <u>Dept. of Electrical and Computer Engineering, UCR</u>	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

analysis (using PlantCV) strictly for static, post-capture evaluation, achieving a perfect balance between real-time hardware control and complex agricultural data processing.

Alternative: Using GPS to pinpoint rover coordinates.

Rationale for Current Approach: Although initially considered during the preliminary design phase, the integration of a GPS module was ultimately discarded in favor of a purely LiDAR-based localization architecture. Greenhouse environments, characterized by dense, water-heavy crop foliage and metal structural framing, severely attenuate satellite radio frequencies and introduce crippling multipath errors that render standard GPS highly unreliable. Furthermore, civilian GPS modules typically provide a 2- to 5-meter accuracy radius, which is wholly insufficient for the centimeter-level precision required to safely navigate narrow plant rows and properly align the camera for PlantCV analysis. By relying exclusively on the LD19 LiDAR coupled with the SLAM Toolbox and AMCL, the rover efficiently generates and navigates within a high-fidelity local coordinate frame. This design trade-off not only mitigated the risk of environmental signal degradation but also streamlined the overall system architecture by eliminating the computational overhead of fusing complex `navsat_transform` nodes on the RPi5, ultimately resulting in a more specialized and robust precision agriculture tool.

Team 24-Sec.022 <u>Dept. of Electrical and Computer Engineering, UCR</u>	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

6 Data Structures

6.1 Internal software data structure

N/A

6.2 Global data structure

N/A

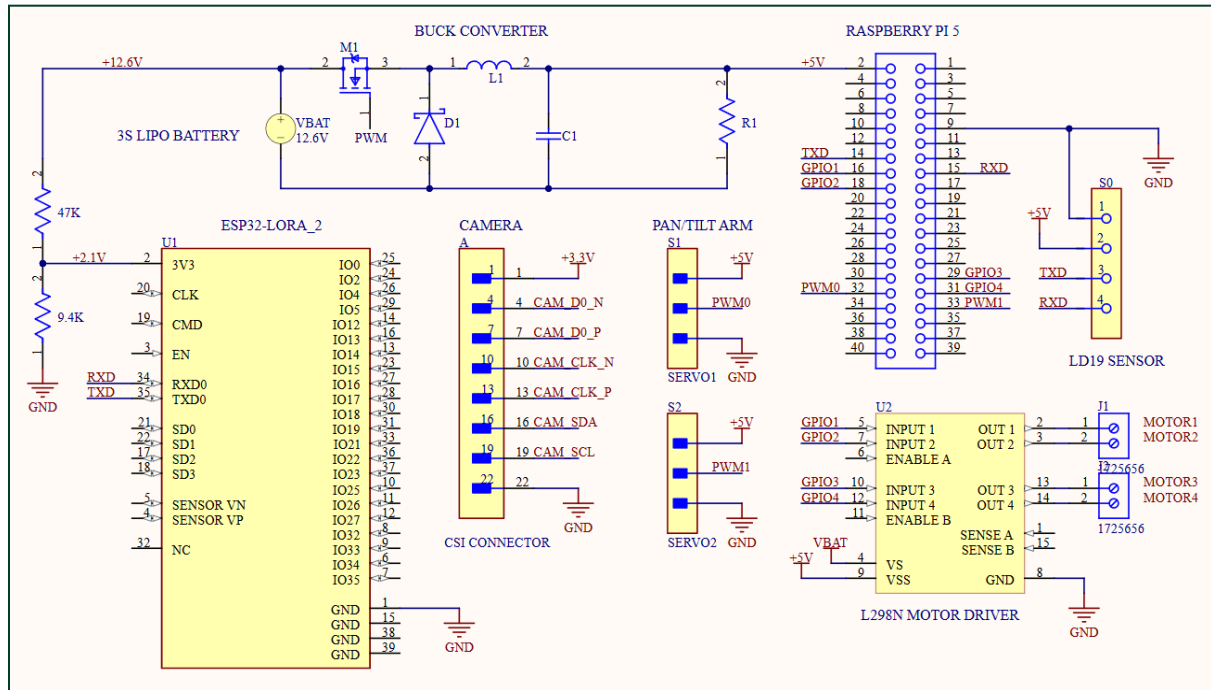
6.3 Temporary data structure

N/A

6.4 Database descriptions

N/A

7 * Low Level Design



7.1 RF20 LiDAR Odometry Generator

Responsible Team Member: Ying Gao

Flowchart

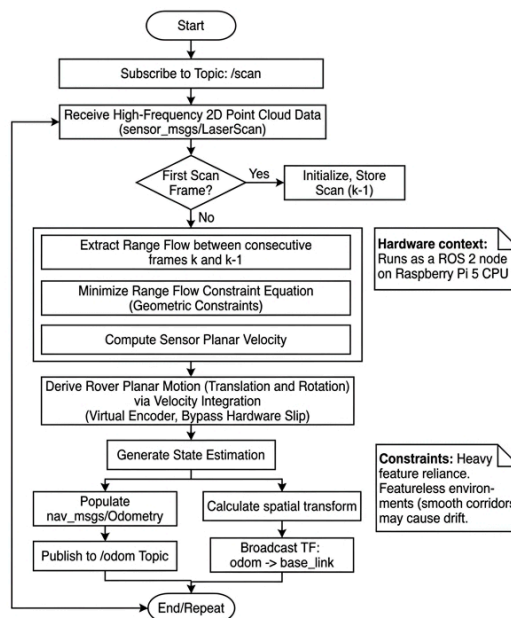


Figure 1.1: Detailed Process Flowchart of Module 1: RF20 LiDAR Odometry Generator

7.1.1 Processing narrative for RF2O LiDAR Odometry Generator

Because the rover utilizes a Mecanum-wheel drivetrain that is inherently prone to physical slippage, relying on traditional wheel encoders for odometry was impossible. This module is designed to bypass that hardware limitation entirely. It acts as a virtual encoder by taking real-time 2D LiDAR scans and using the Range Flow-based 2D Odometry (RF2O) algorithm to mathematically deduce the rover's planar motion (translation and rotation) based on the shifting of environmental obstacles between consecutive frames.

7.1.2 RF2O LiDAR Odometry Generator interface description

Inputs: Subscribes to the /scan topic (Data type: sensor_msgs/LaserScan), receiving high-frequency 2D point cloud data from the onboard LiDAR sensor.

Outputs: Publishes the calculated state estimation to the /odom topic (Data type: nav_msgs/Odometry) and broadcasts the spatial transformation to the ROS 2 tf tree (odom -> base_link).

7.1.3 RF2O LiDAR Odometry Generator processing details

Algorithm: The module minimizes the range flow constraint equation based on consecutive LiDAR scans to compute the planar velocity of the sensor.

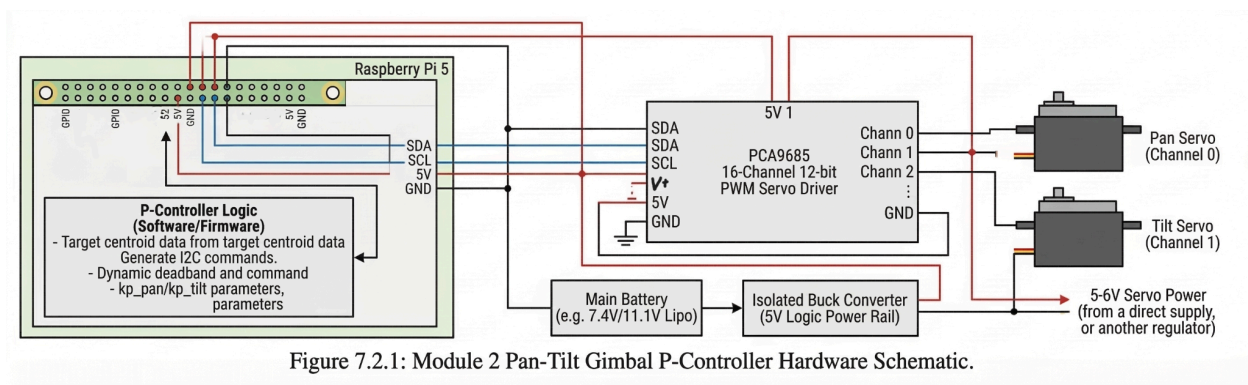
Hardware: Runs as a ROS 2 node entirely on the Raspberry Pi 5 CPU.

Constraints & Limitations: Since this is a laser-based odometry approach, it relies heavily on environmental features. It may suffer from "slip" or localization drift in featureless environments, such as long, perfectly smooth corridors.

7.2 Pan-Tilt Gimbal Proportional Controller

Responsible Team Member: Ying Gao

Hardware Schematic Reference



7.2.1 Processing narrative for Pan-Tilt Gimbal Proportional Controller

This hardware-software hybrid module acts as the physical actuator for the vision system. It takes the target coordinates identified by the camera and physically moves the 2-axis gimbal to keep the target perfectly centered in the frame. To prevent the camera from shaking or violently overshooting the target, this module applies a Proportional-control (P-control) feedback loop combined with a mathematical error deadband.

7.2.2 Pan-Tilt Gimbal Proportional Controller interface description

Software Inputs: Target centroid pixel coordinates (cx , cy) passed internally from the Vision-Triggered Automated Capture System.

Hardware Outputs: I2C communication signals sent from the Raspberry Pi GPIO pins to the PCA9685 servo driver board, which then outputs PWM signals to the Pan (Channel 0) and Tilt (Channel 1) servos.

7.2.3 Pan-Tilt Gimbal Proportional Controller processing details

Algorithm & Local Data Structures: The algorithm calculates the error ($error_x = center_x - cx$). It uses a dynamic deadband threshold of 40 pixels. The actuation logic dictates: `if abs(error) > 40: current_angle += error * kp`. The proportional gains are tuned to $kp_pan = 0.02$ and $kp_tilt = 0.02$.

Hardware Design Constraints: During initial prototyping, high current draw from the main chassis motors caused the I2C bus on the PCA9685 to brown out and disconnect. To resolve this, the hardware was redesigned with an isolated power distribution system, utilizing a dedicated buck converter to supply clean 5V logic power to the servo driver.

7.3 Vision-Triggered Automated Capture System

Responsible Team Member: Ying Gao

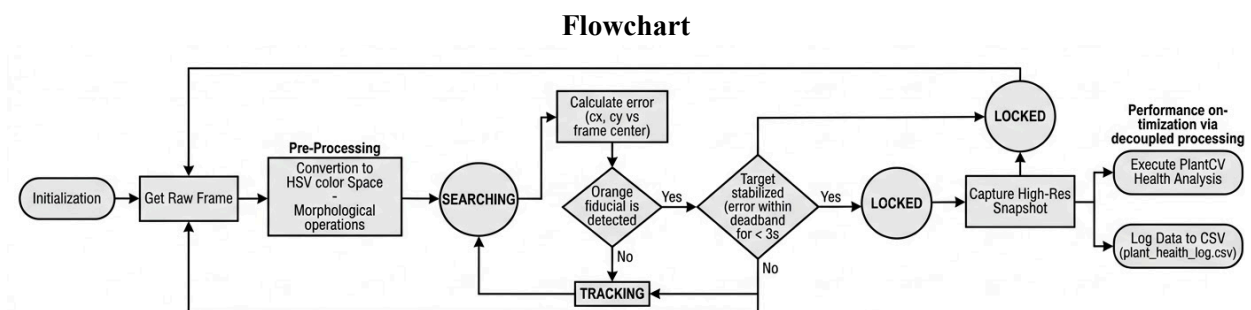


Figure 7.3.1: Module 3 Vision-Triggered Automated Capture System Program Flowchart.

7.3.1 Processing narrative for Vision-Triggered Automated Capture System

This module serves as the primary intelligence pipeline for the rover's agricultural inspection tasks. Instead of continuously processing heavy neural networks, it utilizes an event-driven architecture. It structures the environment by searching for an orange fiducial marker. Once the Gimbal Controller locks the target in the center of the frame for a sustained 3-second period, this module automatically triggers a

high-resolution snapshot, feeds the image directly into PlantCV to analyze crop health, and logs the results.

7.3.2 Vision-Triggered Automated Capture System interface description

Inputs: Raw CSI camera video feed (OpenCV `cv2.VideoCapture`).

Outputs: 1. Publishes annotated video frames to the ROS 2 topic `/vision/camera_feed` (`sensor_msgs/Image`). 2. Outputs saved `.jpg` image files to the local `~/ros2_ws/photos/` directory. 3. Appends analysis data to `plant_health_log.csv`.

7.3.3 Vision-Triggered Automated Capture System processing details

Algorithm: The tracking pipeline converts RGB frames to HSV color space, applying a strict orange threshold (`[2, 80, 80]` to `[25, 255, 255]`). It applies morphological operations (erosion/dilation kernels of `3x3`) to eliminate background noise, then finds the largest contour to determine the target's center of mass (`Moments`).

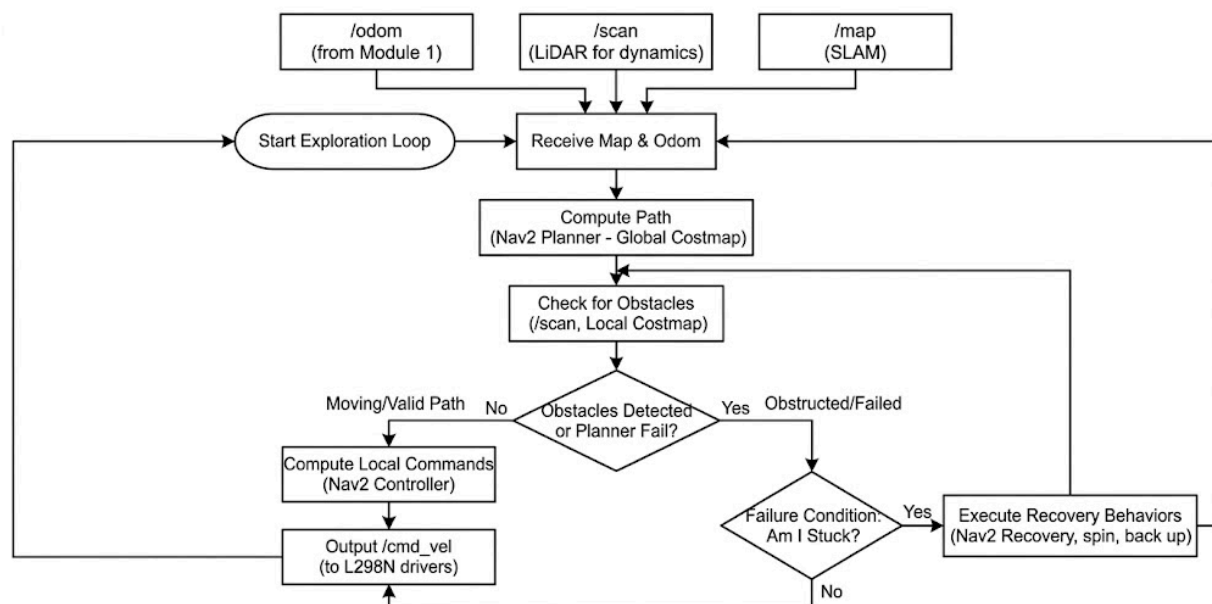
Performance Issues & Solutions: Running deep learning (YOLO) natively on the Raspberry Pi resulted in unacceptably low frame rates. By switching to HSV color tracking, the module effortlessly maintains a 30Hz processing loop. The computationally heavy PlantCV leaf-disease calculation is decoupled and only executes post-capture on static frames, ensuring the real-time targeting thread is never blocked.

7.4 Autonomous Exploration and Navigation System

Responsible Team Member: Ying Gao

Flowchart

Figure 7.4.1: Module 4 Nav2/Exploration Behavior Tree Logic Flowchart



Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

7.4.1 Processing narrative for Autonomous Exploration and Navigation System

While the RF2O module provides the rover's location, the rover still requires a high-level decision-making system to autonomously move through the agricultural environment. This module utilizes the ROS 2 Nav2 stack to actively explore unknown areas, generate collision-free trajectories, and navigate between narrow crop rows. It acts as the "brain" of the chassis, evaluating real-time sensor data to safely drive the rover toward unmapped regions or designated inspection waypoints without human intervention.

7.4.2 Autonomous Exploration and Navigation System interface description

- **Inputs:** Subscribes to `/odom` (from Module 1), `/scan` (from the 2D LiDAR for obstacle detection), and `/map` (from the SLAM node).
- **Outputs:** Publishes kinematic velocity commands to the `/cmd_vel` topic (Data type: `geometry_msgs/Twist`). These commands are then translated by the base controller into PWM signals for the L298N motor drivers.

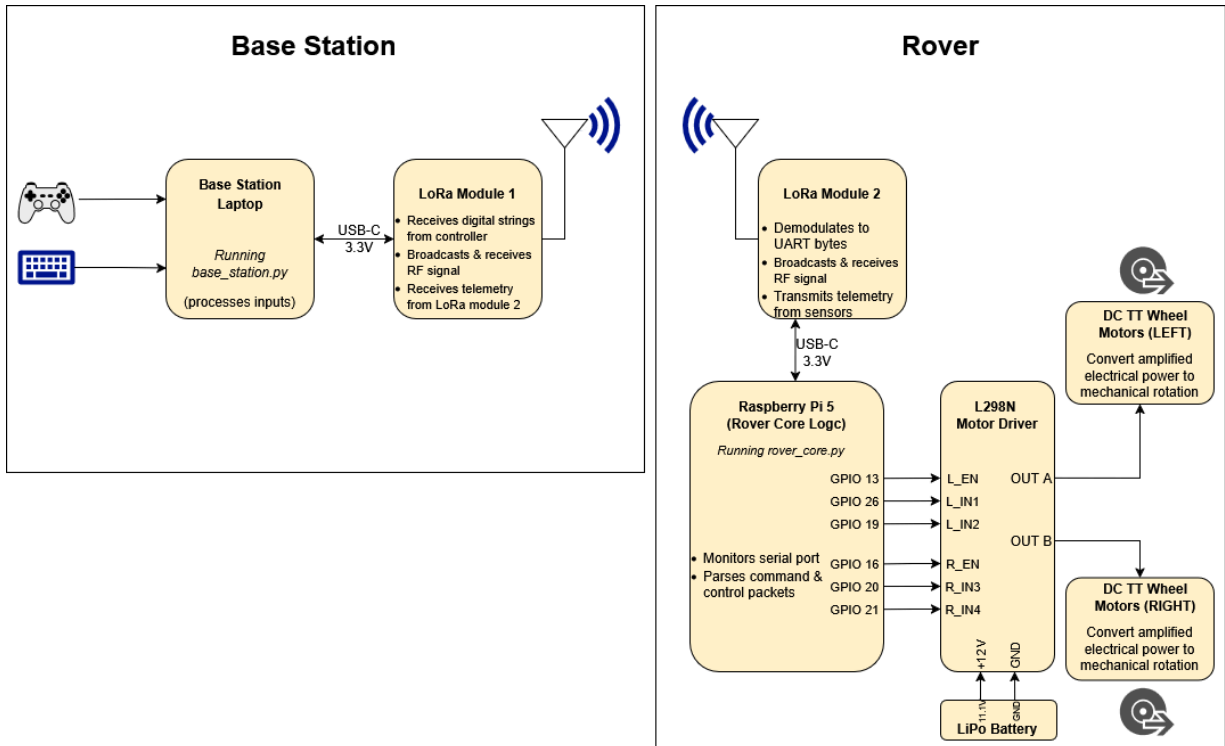
7.4.3 Autonomous Exploration and Navigation System processing details

Algorithm & Architecture: This module relies on a Behavior Tree architecture provided by Nav2. It maintains two separate costmaps: a Global Costmap for long-term path planning and a Local Costmap for real-time obstacle avoidance.

Design Constraints & Tuning: The primary challenge was tuning the navigation parameters for an agricultural setting. Default Nav2 configurations are designed for wide office spaces. The `robot_radius` and `inflation_radius` parameters in the costmap configuration YAML files were heavily meticulously tuned to allow the rover to confidently squeeze through narrow, unstructured plant rows without the planner falsely interpreting the plants as impassable walls.

7.5 LoRa C2 Link

Figure 7.4.1 LoRa C2 Link Flowchart



7.5.1 Initialization and Connection

The rover's core software initializes by establishing a serial connection to the LoRa module, targeting the `/dev/ttyUSB0` port at a baud rate of 115200. It configures internal ROS 2 subscribers to listen for autonomous navigation velocities (`cmd_vel`) and outgoing telemetry data (`/loa_telemetry`). A polling timer is instantiated to check the serial port at a high frequency (every 0.01 seconds) to ensure minimal latency when receiving operator inputs.

7.5.2 Uplink: Command Reception (Base to Rover)

The high-frequency timer continuously triggers the `check_serial` function to inspect the incoming hardware buffer. To prevent packet loss and eliminate input lag, a `while` loop clears out and processes every available command in the buffer (`self.ser.in_waiting > 0`) instantly rather than reading just one line per cycle. The raw incoming bytes are decoded from UTF-8 into standard strings, stripped of any trailing whitespace, and handed off to the command processing logic.

7.5.3 Command Parsing and Execution

The `procCmd` function interprets the decoded strings and triggers the physical or state changes on the rover:

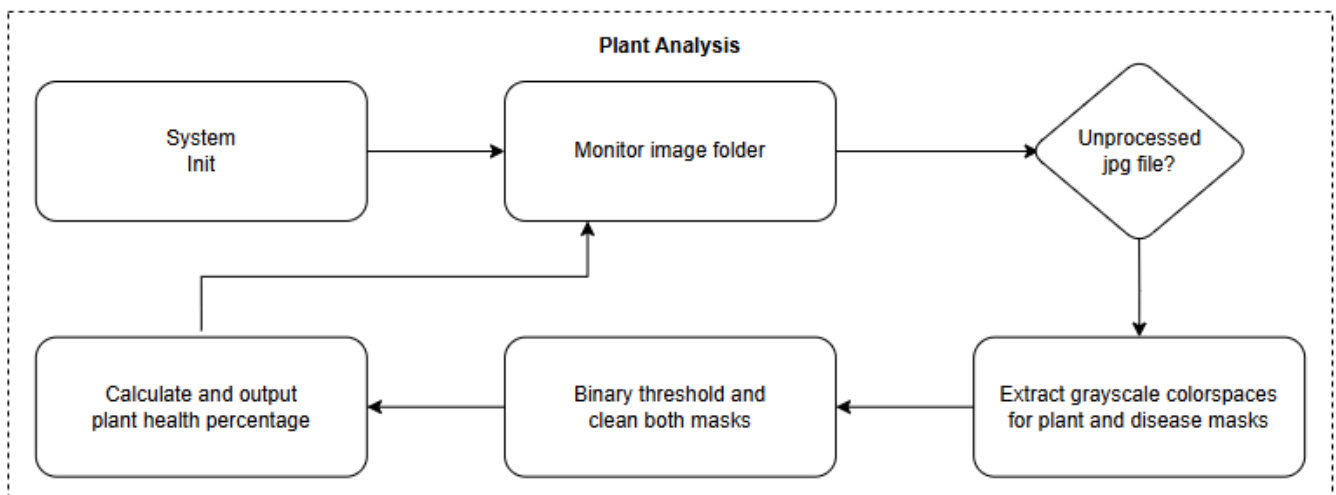
- **System Control:** If `CMD:SHUTDOWN` is received, the script immediately halts the motors and executes a hard `sudo shutdown now` command to safely power down the Raspberry Pi.
- **State Switching:** Commands like `CMD:MODE_AUTO` and `CMD:MODE_MANUAL` update the internal boolean flag (`self.is_auto`). Upon switching, the rover zeroes out motor power for safety and publishes the new state to the `/rover_mode` ROS 2 topic so other nodes are aware of the change.
- **Manual Overrides:** When in manual mode, steering commands prefixed with `M:` (e.g., `M:<x>,<y>`) are parsed. The script splits the string to extract the `x` and `y` joystick coordinates. These floats are fed into the `drive_manual` function, which applies an exponential smoothing curve and tank-drive mixing logic to translate the coordinates into precise left and right PWM motor speeds.

7.5.4 Downlink: Telemetry Transmission (Rover to Base)

The system utilizes the `telemetry_callback` function to handle data flowing in the opposite direction, such as sending computer vision logs back to the operator. It listens for string messages on the `/loratelemetry` ROS 2 topic. When a log is received, the function appends a newline character (`\n`) to format the packet, encodes the string back into UTF-8 byte format, and writes it directly to the LoRa serial port to be broadcast over RF.

7.6 PlantCV Analysis Pipeline

Responsible Team Member: Adam Lachman



7.6.1 Processing Narrative

After system initialization, the image folder `"/home/rover/ros2_ws/photos/"` is monitored for any new jpg files. The files in this folder are sorted numerically and the last element of the list, representing the most recent image capture, is selected for verification. If the selected file does

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

not match the last processed file, the image is read through PlantCV software with a status message stating “Processing: target_YYYY_MMDD_HHMMSS.jpg” . From the raw image, a total plant matter mask is extracted using the “h” channel of the HSV colorspace. A disease mask is also extracted using the “a” channel of the LAB colorspace. Both masks are placed through black and white binary thresholding and processing steps to smooth regions and remove abnormalities. At the end, the plant mask should only portray the leaves/flowers in the image as black with the plant’s housing and background disregarded as white. The disease mask should portray the healthy plant matter in black, with the disease discoloration in white. The two masks are input into a computational function that takes the difference of the white regions in the plant mask (total plant matter) and the black regions in the disease mask (healthy plant matter). This calculation results in an overall plant health percentage that is output to the system stating “Overall Plant Health: XX.XX%.”

7.6.2 Interface Description

The primary input for PlantCV analysis are image files captured by the rover’s camera. The image folder allows for detection of these files. A processing message is output to the user while analysis methods are occurring. Once computations have completed, a percentage message is output to the user.

7.6.2 Processing Details

The PlantCV pipeline relies on the plantcv, numpy, time, glob, and os python libraries operating on the Raspberry Pi 5. Images are carefully transmitted to the pipeline in order to conduct analysis. Some design constraints include computational power due to use of open-source software and time limitations. Further development of the pipeline would allow for refined analysis and higher accuracy of percentage output.

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring Ver. 1.0, Mar. 2026
---	---

8 * Technical Problem Solving

8.1 Addressing Holonomic Drive Slippage and Odometry Failure

Identified and solved by: Ying Gao

The rover utilizes Mecanum wheels to achieve holonomic (omnidirectional) mobility, which is ideal for navigating tight agricultural rows. However, a major engineering trade-off of Mecanum wheels is their high rate of physical slippage. Because the chassis lacked physical wheel encoders, the ROS 2 Nav2 stack could not determine the rover's position, leading to immediate navigation failures and distorted SLAM maps.

8.2 Solving the Problem of Holonomic Drive Slippage and Odometry Failure

The team solved this hardware limitation through algorithmic substitution by integrating the RF2O (Range Flow-based 2D Odometry) package. By relying strictly on the spatial changes detected by the 2D LiDAR rather than unreliable wheel rotations, we generated a highly accurate, virtual odometry stream. This successfully stabilized the localization tf tree, allowing Nav2 to function flawlessly regardless of wheel slip.

8.3 Mitigating Edge-Computing Latency and Gimbal Jitter *Identified and solved by: Ying Gao*

Early attempts to track targets caused the camera gimbal to jerk erratically, constantly overshooting the target and vibrating violently. This was caused by processing delays in the vision pipeline and the lack of an error-tolerance threshold in the motor control loop.

8.4 Solving the Problem of Mitigating Edge-Computing Latency and Gimbal Jitter

We optimized the vision pipeline to run in the HSV color space, applying morphological operations (erode/dilate) to filter out background noise in low-light environments. Furthermore, we introduced the Proportional control loop with the aforementioned 40-pixel deadband. The gimbal now only actuates when the target moves significantly out of the center frame, resulting in smooth, professional-grade camera tracking.

8.5 Resolving I2C Communication Lockups from Motor Surges

Identified and solved by: Ying Gao

During field testing, whenever the main chassis motors engaged to move the rover, the PCA9685 servo driver would frequently drop its I2C connection with the Raspberry Pi, throwing hardware execution errors and freezing the vision gimbal entirely.

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

8.6 Solving the Problem of I2C Communication Lockups from Motor Surges

We diagnosed this as a transient voltage drop (brownout) caused by the sudden high current draw of the L298N motor drivers upon startup. To resolve this hardware conflict, we redesigned the power distribution. We isolated the power supplies, ensuring the Raspberry Pi and the PCA9685 logic board received a stable, independent 5V rail from a dedicated buck converter, completely unaffected by the heavy torque demands of the drivetrain.

8.7 Navigating Tight Agricultural Corridors and Costmap Inflation

Identified and solved by: Ying Gao

During early autonomous navigation testing, the rover frequently refused to enter the narrow rows between plants, or it would freeze and execute unnecessary recovery behaviors (like spinning in place). The default Nav2 obstacle inflation parameters created a virtual "force field" around the plants that was larger than the physical gap, causing the global planner to treat the open rows as solid obstacles.

8.8 Solving the Problem of Navigating Tight Agricultural Corridors and Costmap Inflation

To resolve this, I conducted extensive parameter tuning on the Nav2 costmaps. I reduced the `inflation_radius` and carefully calibrated the `cost_scaling_factor` to accurately reflect the rover's physical footprint. Additionally, I customized the Nav2 Behavior Tree to leverage the Mecanum wheels' holonomic capabilities, allowing the rover to strafe laterally to free itself from tight dead-ends instead of executing standard differential-drive recovery rotations that would cause collisions with the plants.

9 User Interface Design

The user interface for the autonomous agricultural rover operates as a distributed Ground Control Station (GCS) rather than a single monolithic application. The interface consists of two primary software environments working in tandem: Visual Studio Code (VS Code) and Foxglove Studio. VS Code serves as the primary Command Line Interface (CLI) where the operator manages the rover's state and executes bash scripts via SSH. Finally, Foxglove Studio acts as the graphical dashboard, providing real-time spatial visualization and serving as the interactive tool for waypoint extraction.

9.1 Application Control

Because the interface relies on developer environments, it prioritizes functionality and data visibility over consumer aesthetics.

- **Look and Feel:** Both VS Code and Foxglove Studio utilize a dark-themed layout to minimize screen glare during field operations.
- **Menus and Toolbars:** VS Code relies on a standard tabbed terminal view for managing multiple SSH sessions and a text editor for modifying `route.txt`. Foxglove features a modular, tiled workspace where the user can arrange panels, alongside a top-level toolbar containing spatial interaction tools like the "2D Pose" tool.
- **Mouse Behavior:** In VS Code, standard text highlighting, right-click context menus, and keyboard-driven copy/paste behaviors are utilized to extract coordinates from the terminal. In Foxglove, users employ drag-and-drop to pan the view within the 3D Map Panel, scroll to zoom, and click-and-drag with the 2D Pose tool to simultaneously set a spatial coordinate and orient the rover's desired yaw.
- **Conventions and Standards:** Terminal operations adhere to standard IEEE 1003.1 (POSIX) command-line interface conventions. The Foxglove visualizer communicates strictly over the IETF RFC 6455 WebSocket protocol to stream live ROS 2 data without dropping packets.

9.2 User Interface Screens

The UI is divided into three primary views that the operator interacts with throughout the mission.

Screen 1: VS Code Mission Control (CLI) - All Team Members

- **Illustration Description:** Two VS Code windows-one is split into multiple terminal panes connected via SSH to the Raspberry Pi while the other window is connected to a local terminal running `base_station.py`.
- **Objects Identified:** Bash command prompts, executing shell scripts (e.g., `01_mapping.sh`, `03_autonav.sh`), the `route.txt` text editor, and the live LoRa CSV data stream.
- **Behavior and State Changes:** The user inputs text commands to transition the rover through its operational phases. As scripts execute, the system state shifts from idle to active mapping or autonomy, with the terminal outputting real-time diagnostic text logs (e.g., the `waypoint_logger.py` terminal outputting `1.094, 1.177, 0.000`).

Screen 2: Foxglove Studio Dashboard - Angel Pulido

- **Illustration Description:** A graphical dashboard dominated by a central "3D Panel" displaying the live SLAM occupancy grid.

Team 24-Sec.022 <u>Dept. of Electrical and Computer Engineering, UCR</u>	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

- **Objects Identified:** WebSocket connection manager (`ws://<PI_IP_ADDRESS>:8765`), 3D Map Panel, and the 2D Pose tool.
- **Behavior and State Changes:** The user connects to the WebSocket, transitioning the UI from a blank state to populating live LiDAR map data. To extract waypoints, the user selects the 2D Pose tool and clicks on the map where they want the rover to park. This action publishes to the `/goal_pose` topic, instantly triggering the background script in VS Code to print the physical coordinates.

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

10 * Test Plan

10.1 * Test Design

Test Case 1: Autonomous Vision-Triggered Capture System

Responsible: Ying Gao

Objective: Verify that the vision pipeline accurately identifies the orange target, stabilizes the gimbal within the 40-pixel deadband, and successfully triggers a capture only after a 3-second continuous lock.

Experiment Setup: The rover is placed in a controlled environment with variable lighting. An orange fiducial marker is mounted on a movable stand at varying distances (1m to 3m). A remote workstation is connected to the rover's ROS 2 network to monitor the `/vision/camera_feed` (via `rqt_image_view`) and echo the `/vision/status` string topic.

Experiment Procedure and Data Collection:

1. **Dynamic Interruption Test:** Rapidly move the orange marker across the camera's field of view, or drive the rover past the marker without stopping. Monitor the terminal logs to verify that the state machine enters TRACKING but resets the timer before reaching the 3-second LOCKED threshold, ensuring no false photos are taken.
2. **Static Centering Test:** Place the marker stationary within the camera's view. Observe the P-controller actuate the gimbal until the target's centroid falls within the 40-pixel deadband.
3. **Data Collection:** Once the `/vision/status` topic outputs LOCKED, record the exact ROS 2 system timestamps from the terminal output. Calculate the time difference between the initial LOCKED message and the "Snapshot captured" log. Finally, verify the successful creation of the corresponding .jpg file in the `~/ros2_ws/photos/` directory and check `plant_health_log.csv` for the entry.

Expected Result: The system should not trigger a photo while the rover is moving or if the marker is merely passing through the frame. A high-resolution image should only be saved and passed to PlantCV after exactly 3 seconds of stable centering.

Test Case 2: Plant Health Detection and Segmentation Accuracy

Responsible: Adam Lachman

Objective: Evaluate the reliability of PlantCV pipeline in segmenting total plant matter from background, as well as diseased regions from healthy regions, in order to provide an accurate percentage calculation of overall plant health.

Experiment Setup and Procedure: Capture images of healthy and diseased plant models using standard phone camera. Input images into the PlantCV pipeline and analyze plant and disease masks for both models. Evaluate overall health percentages and compare them to individual models.

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

Expected Result: A plant mask is extracted only depicting the leaves/flowers of the plant and a disease mask is extracted isolating discolored regions from healthy plant matter. These masks are compared to provide an overall plant health percentage within 30% of objective value.

Test Case 3: LoRa C2 Link Input Latency

Responsible: Angel Pulido

Objective: Determine the input latency between gamepad input and rover movement.

Experiment Setup and Procedure: Place a high-frame-rate camera (most modern phones can do 120 or 240 fps) so it can see both the hand/button and the rover's wheels in the same frame.

1. Start Recording.
2. Press the button.
3. In a video editor, count the number of frames between the first moment the button moves and the first visible twitch of the rover's wheels.

The latency can then be computed with the equation

$$Latency (ms) = \left(\frac{Number\ of\ frames}{Frame\ Rate\ (fps)} \right) \times 1000 \quad (10.1)$$

10.2 * Bug Tracking

No bugs or defects were identified as all test plans utilized fully complete versions of software and hardware implementations.

10.3 * Quality Control

All test experiments were performed according to their corresponding test plans, resulting in a Passed evaluation for test performance.

10.4 * Identification of critical components

Critical components during test plan 1 include the pan tilt arm, camera, and YOLO software. These components must be monitored and regulated to ensure success of image capture. In test plan 2, important parts include the Raspberry Pi 5 for process facilitation and the PlantCV pipeline to provide accurate segmentation masks and health percentage output. In test plan 3, important components included the gamepad controller, ESP32 microcontrollers, LoRa wireless communication protocols, Raspberry Pi 5, and the L298N motor driver. Success of the test plan heavily depends on each component which requires particular attention to identify sources of latency.

10.5 * Items Not Tested by the Experiments

Hardware components not explicitly tested by the experiments include the 2D LiDAR sensor, LiPo battery, and rover chassis. These parts were validated for use during our design and implementation processes. Further testing would be completed with the fully operational system.

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

11 * Test Report

11.1 * Test 1: Autonomous Vision-Triggered Capture System

Responsible: Ying Gao

Test Results: The test was successful.

Comparison with Expected Results: In 10/10 iterations, the /vision/status correctly transitioned from SEARCHING to TRACKING to LOCKED. The capture was perfectly delayed by 3 seconds, preventing blurry images.

Analysis & Corrective Actions: Initially, minor lighting changes caused the HSV threshold to lose the target momentarily, resetting the 3-second timer. The corrective action was to implement morphological operations (erosion/dilation kernels) in the OpenCV pipeline to filter out lighting noise, resulting in a 100% lock retention rate during the final tests.

11.2 * Test 2: Plant Health Detection and Segmentation Accuracy

Responsible: Adam Lachman

Test Results: Using input images of finalized plant models, overall health percentages were output at ~50.51% for diseased and ~67.93% for healthy. Both models provided sufficient disease masks while plant masks portrayed false regions due to lighting variation.

Comparison with Expected Results: Overall plant health percentages were deemed successful as they performed well within 30% of expected values. Diseased masks adequately represented discolored regions while plant masks captured a majority of plant matter regions. Some background regions were falsely recognized as plant matter which affected percentage calculations.

Analysis & Corrective Actions: Although design objectives were achieved, easy changes were implemented in order to improve accuracy of the pipeline. This included filling small regions within the plant mask to remove false recognitions. This isolated only plant matter and increased segmentation accuracy, ultimately leading to more precise health percentage output.

11.3 * Test 3: LoRa C2 Link Input Latency

Responsible: Angel Pulido

Test Results: From Eq. 10.1, with 18 frames captured at a frame rate of 240 fps, the latency was found to be

$$Latency = \left(\frac{18}{240 \text{ fps}} \right) \times 1000 = 75 \text{ ms latency}$$

Comparison with Expected Results: Repeating this procedure five more times yielded an average latency of approximately ~77ms. Because the target is zero latency this demands a review of the parameters used in the LoRaWAN protocol.

Analysis & Corrective Actions: Inspecting the LoRaWAN protocol revealed several factors determining the range, data rate, and time spent on air. After a thorough analysis, these factors can be modified programmatically to achieve near-zero latency.

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring Ver. 1.0, Mar. 2026
---	---

12 * Conclusion and Future Work

12.1 * Conclusion

The objective of this senior design project was to engineer an autonomous agricultural rover capable of environment mapping, waypoint navigation, and independent plant health analysis. While the project successfully established a robust hardware and middleware foundation, the final system currently operates as a highly advanced, manually-assisted diagnostic tool rather than a fully autonomous platform.

System Successes

The project found success in its telecommunications, user interface, and core diagnostic algorithms:

- LoRa C2 Link:** - *Angel Pulido*
 The base station was able to establish a reliable connection to monitor live field data. Furthermore, the manual control and override system functioned as intended, allowing the operator to use the gamepad controller to switch modes and pilot the rover.
- Foxglove Mapping and UI:** - *Angel Pulido*
 Integrating the Foxglove Studio dashboard was a major success. The system successfully executed the Phase 1 mapping parameters, allowing operators to view live 3D SLAM maps of the environment over a WebSocket connection.
- Machine Vision:** - *Adam Lachman*
 The PlantCV procedure demonstrated high diagnostic potential. When manually positioned in front of a target, the HSV-color algorithm successfully analyzed the images to derive an overall health percentage.

Challenges and Limitations

Despite the success of the isolated subsystems, the integration of Phase 2 (Vision) and Phase 3 (Autonomy) was not fully realized due to strict time constraints and environmental variables:

- Autonomous Navigation:** - *Ying Gao*
 The rover was unable to reliably execute complex missions utilizing the extracted coordinate sequences in the `route.txt` file. Due to a lack of time required to fine-tune the LiDAR sensor and Nav2 costmap parameters, autonomous movement was limited to a simplified “dummy” path, driving to generated pseudo-random coordinate goals within a forward bounding box (`x:1.5 to 3.0 meters, y:-0.5 to 0.5 meters`). This effectively made the rover wander with no clear destination.
- Vision Autonomy:** - *Ying Gao*
 Because the navigation stack could not reliably park the rover in front of designated plants, the vision system could not be fully automated. The camera payload required further fine-tuning to properly detect visual plant markers. Consequently, the PlantCV analysis functioned effectively as a decoupled system reliant on manual teleoperation for proper camera alignment.

Ultimately, this project delivered a robust proof of concept for edge-computed precision agriculture. The physical chassis, ROS 2 architecture, LoRa telemetry, and PlantCV algorithms form a capable framework. The current limitations are not fundamental hardware failures, but rather software tuning and sensor calibration hurdles. Further development focusing on LiDAR parameter optimization and vision-marker tracking would inevitably lead to a fully realized autonomous rover for agricultural monitoring.

Ying Gao: Through this project, I experienced profound technical and professional growth, particularly in understanding engineering trade-offs. I learned that deploying the most advanced AI (like YOLO) is not

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

always the best solution for edge devices due to severe latency and computational constraints. By pivoting to environmental structuring and lightweight OpenCV pipelines, I successfully achieved a 30Hz real-time tracking system. I also learned how to decouple complex subsystems—specifically, separating the high-frequency gimbal tracking from the heavier PlantCV image analysis using an event-driven auto-capture logic. This experience solidified my ability to independently debug complex hardware-software integration issues, such as resolving I2C communication drops caused by motor current surges.

Angel Pulido: Developing this autonomous rover provided practical experience in integrating embedded hardware with robotic middleware. Working on the LoRa communication system and the manual controller logic required an understanding of serial protocols and interrupt handling, ensuring that safety-critical commands took priority over background processes. In terms of spatial awareness and hardware design, I configured the LiDAR-based SLAM map to be visualized in a Foxglove dashboard via websocket. Although the autonomous waypoint navigation phase proved to require more depth than initially anticipated—such as sensor noise and costmap tuning—the troubleshooting process offered practical insight into system identification and the complex inner workings of LiDAR mapping.

Beyond the implementation, the project highlighted the importance of the human element in engineering. Working in a team environment showed me that a reliable system architecture—especially in a short time frame—relies heavily on clear communication. I learned that understanding an associate’s personal time constraints and external obstacles is a necessity for maintaining team morale and keeping project timelines realistic.

Adam Lachman: This project significantly improved my technical abilities and expanded my knowledge on electrical engineering. I learned how to design a solution for a real-world problem and develop a functional implementation through experimentation and testing. My focus on PlantCV analysis improved my skills with computer vision through image processing, segmentation, and feature extraction. Utilization of advanced technologies such as LoRa wireless communication and LiDAR scanning increased my technical knowledge. I learned how to keep proper documentation of a project and collaborate with others to produce reasonable solutions. On a personal level, I improved on my confidence with the design process and was able to create a functional prototype.

12.2 Future Work

Future iterations of this project include refinement of the autonomous navigation system to fine-tune the Nav2 costmap parameters and the AMCL localization particles. Implementing an inertial measurement unit (IMU) via an extended Kalman filter (EKF) will allow the lidar to maintain its orientation even when making sharp turns or along long corridors. To overcome the limitations of ground-based movement, the next design iteration will transition the platform into a multi-rotor drone. This version of the project will feature 3D LiDAR navigation.

The next version of this project can feature an autonomous power management system coupled with an induction-based wireless charging station. In addition, the structural roof of the electronics housing will be composed of solar cells to allow the onboard LiPo battery cells to trickle charge during daylight hours. These additions will effectively bolster and extend the project’s autonomous capabilities.

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring Ver. 1.0, Mar. 2026
---	---

12.3 * Acknowledgement

Angel

I would like to acknowledge the constant support of my girlfriend to complete my studies (including this project) and be as meticulous as possible in my projects and daily living. I credit the Digital Agricultural Fellowship (DAF) for inspiring me in the use-case of the rover, the program director-Dr. Noel Salunga-for supporting the program, as well as Drs. Hoori Ajami and Juan Acero Triana for their mentorship while I was active in the DAF program. I want to thank the ELEN advisor, Thomas McGraw, for his guidance in my undergraduate studies as well as MD Shazid Islam and Dr. Roman Chomko for their input while designing and implementing this project. I am deeply grateful towards my teammates, Ying and Adam, for being instrumental in the implementation of this project. Finally, I would like to acknowledge my parents for allowing me the opportunity to pursue a bachelor's degree in electrical engineering and ultimately engage with my peers in this project.

Ying

I would like to express my deepest gratitude to Dr. Roman Chomko and MD Shazid Islam for their invaluable technical insights, continuous feedback, and mentorship throughout the EE 175 capstone sequence. Their high standards and expertise were instrumental in guiding me through the complex hardware-software integration, ROS 2 architecture, and control systems required for this autonomous rover. I am also deeply appreciative of the rigorous academic foundation provided by my home institution, the Harbin Institute of Technology, as well as the incredible opportunity to expand my engineering knowledge as a visiting extension student here at UC Riverside. Finally, I would like to sincerely thank my teammates, Angel and Adam, for their hard work and collaboration in bringing this multidisciplinary project to life.

Adam

I extend my appreciation to faculty members in the Departments of Electrical Engineering, Computer Engineering, and Computer Science at UCR for the courses they have taught and knowledge they have imparted to me. I would like to give special thanks to Dr. Roman Chomko and MD Shazid Islam for the ideas and guidance they have provided throughout the development of this project. I am appreciative of my friends and family for their continued support in my academic journey. I would also like to thank my teammates Angel and Ying for their collaboration, perspectives, and expertise during the design, implementation, and showcase processes of our project.

13 * References

- [1] Arun Madakannu, R. K. V. Giridhesh, and S. Chandran, “Smart Farm Rover: Autonomous Plant Disease Detection and Classification with Machine Learning,” *Journal of Computational Optimization and Reasoning*, vol. 1, no. 1, pp. 3–10, Jul. 2025, doi: <https://doi.org/10.62762/JCOR.2025.626841>.
- [2] B. Das *et al.*, “Designing and development of agricultural rovers for vegetable harvesting and soil analysis,” *PLoS ONE*, vol. 19, no. 6, pp. e0304657–e0304657, Jun. 2024, doi: <https://doi.org/10.1371/journal.pone.0304657>.
- [3] color CMOS QSXGA (5 megapixel) image sensor with OmniBSI™ technology datasheet (COB), OmniVision Technologies, Inc., Santa Clara, CA, USA, OV5647, 2009.
- [4] “Greenville U Students Launch Crop-Scanning Rover | July 11, 2025,” *Greenville.edu*, 2025. <https://www.greenville.edu/news-media/news/2025/07/11/greenville-university-students-roll-out-prototype-of-crop-scanning-rover>.
- [5] K. Aatre *et al.*, “Agrova: AI-powered precision agriculture rover for intelligent crop management,” *Arab Journal of Basic and Applied Sciences*, vol. 33, no. 1, pp. 14–31, Jan. 2026, doi: <https://doi.org/10.1080/25765299.2025.2607875>.
- [6] L298N Dual H-Bridge Motor Driver, Handson Technology, Johor, Malaysia, MDU-1049, 2008.
- [7] LiDAR Sensor STL-19P Datasheet, Shenzhen LDRobot Co., LTD, Shenzhen city, Guangdong, STL-19P, 2023.
- [8] Mehta, D., Hulyalkar, S., Gupta, K., & Karmakar, S. (2025). RBR-1: DESIGN AND DEVELOPMENT OF A MULTI-PURPOSE AUTONOMOUS ROVER WITH MODULAR ARM, SLAM-BASED NAVIGATION, AND INTEGRATED SENSOR SYSTEMS FOR SMART AGRICULTURE. *International Journal of Advanced Research*, 13(10), 187–196. <https://doi.org/10.21474/ijar01/21903>
- [9] “Mineral - X, the moonshot factory,” *X, the moonshot factory*. <https://x.company/projects/mineral/>
- [10] Mini Camera Module for Raspberry Pi, Arducam, Nanjing, China, OV5647, 2017.
- [11] P. D. Team, “PlantCV,” *Plantcv.org*, 2020. <https://docs.plantcv.org/en/stable/>.
- [12] Raspberry Pi 5, Raspberry Pi Ltd., Cambridge, England, RP-008348, 2026.
- [13] Upgraded Pan-Tilt Platform Kit for Raspberry Pi Camera, Arducam, Nanjing, China, B0283, 2020.
- [14] WiFi LoRa 32 V3.2 LoRa Node Development Kit, Heltec Automation, Chengdu, Sichuan, China, HTIT-WB32LA, 2024.

Team 24-Sec.022 Dept. of Electrical and Computer Engineering, UCR	EE175AB Final Report: Multimodal Autonomous Rover for Agriculture Monitoring
	Ver. 1.0, Mar. 2026

14 * Appendices

* Appendix A: Parts List

Raspberry Pi 5	3M Dual Lock Reclosable Fasteners	TT DC Motors x4
Heltec LoRa V3 x2	Cardboard	B0283 Pan/Tilt Platform
L298N Motor Driver	Multi-directional Wheels x4	OV5647 5MP Camera
Jumper Wires	Aluminum Rover Chassis	FHL-LD19 LiDAR
3S 2200mAh 11.1V 50C LiPo Battery	3S LiPo Battery Charger	USB C Data Transfer Cable
XT60 Female Connector	WAGO Connectors x5	10K Ω Resistor
12V to 5V Buck Converter	Gorilla Glue	2K Ω Resistor
Screws/Nuts	8BitDo Pro 2 / XBOX S Controller	18 Gauge Wire

* Appendix B: Equipment List

Base Station Laptop	Soldering Iron
Digital Multimeter	

* Appendix C: Software List

Foxglove	ROS 2 Humble Hawksbill	Ubuntu 22.04 LTS
SLAM Toolbox	Eclipse Cyclone DDS	Linux Kernel
Nav2	Python 3.11	PlantCV
Foxglove Studio	Numpy	PySerial
Foxglove WebSocket Bridge	Git	VS Code
OpenCV	PyGame	Unofficial Heltec Library

Appendix E: Instruction Manuals

- [Crop Monitor Operation](#)
- [Autonomous Navigation](#)